Base de datos a Datos: Especial Join

David Valbuena Segura | Curso 2025-2026

10 de diciembre de 2025
Especial Consultas.

1 MANUAL DIDÁCTICO: JOINS EN MySQL 5 1.1 SECCIÓN 1: FUNDAMENTOS DE JOINS . . . . . 5 1.2 SECCIÓN 2: INNER JOIN - COMPOSICIONES INTERNAS . . . . . 8 1.3 SECCIÓN 5: JOINS ESPECIALES . . . . . 14 1.4 SECCIÓN 6: JOINS AVANZADOS . . . . . 22 1.5 SECCIÓN 7: JOIN VS SUBCONSULTAS . . . . . 28 1.6 SECCIÓN 8: ERRORES COMUNES Y SOLUCIONES . . . . . 33 1.7 SECCIÓN 9: BUENAS PRÁCTICAS Y TABLA DE DECISIÓN . . . . . 38 1.8 SECCIÓN 10: EJERCICIOS PRÁCTICOS CLASIFICADOS . . . . . 43 1.9 SECCIÓN 11: CASOS PRÁCTICOS DE NEGOCIO . . . . . 55 1.10 SECCIÓN 12: SCRIPT MySQL COMPLETO Y VERIFICADO 65

Módulo: 0484 - Bases de Datos
Ciclo Formativo: 1º DAM (Desarrollo de Aplicaciones Multiplataforma)
Autor: David Valbuena Segura
Duración: 8-10 horas lectivas
Resultado de Aprendizaje: RA3 - Criterios 3c y 3d (25%)

Descripción

Manual didáctico completo sobre **JOINs y composiciones de tablas en MySQL**, diseñado específicamente para estu diantes de primer curso de DAM sin conocimientos previos del tema. El manual sigue una metodología progresiva desde conceptos básicos hasta casos reales complejos de negocio, utilizando una base de datos de biblioteca digital como hilo conductor.

Objetivos del Manual

- Comprender qué son los JOINs y por qué son necesarios en bases de datos relacionales
- Dominar INNER JOIN para composiciones internas entre tablas
- Aplicar LEFT JOIN y RIGHT JOIN para composiciones externas
- Conocer JOINs especiales: CROSS JOIN y SELF JOIN
- Combinar JOINs con agregaciones y subconsultas
- Comparar JOINs vs subconsultas y elegir la mejor opción
- Identificar y solucionar errores comunes
- Resolver casos reales de análisis de negocio

Estructura del Manual

Parte Teórica (Secciones 1-8)

1. **Fundamentos de JOINs**
    - Concepto y necesidad de JOINs
    - Claves primarias y foráneas como «puentes»
    - Panorama de tipos de JOIN
    - Base de datos de ejemplo: BIBLIOTECA_DIGITAL (7 tablas)
2. **INNER JOIN - Composiciones Internas**
    - Sintaxis moderna vs sintaxis antigua
    - Uso de alias de tablas
    - JOINs con múltiples tablas (3, 4, 5 tablas)
    - Diferencia entre ON y WHERE
    - 9 ejemplos verificados con resultados

3. **LEFT JOIN - Composiciones Externas Izquierdas**
    - Concepto de «todos los registros de la izquierda»
    - Manejo de valores NULL
    - Patrón LEFT JOIN + IS NULL (encontrar «sin coincidencias»)
    - COUNT(*) vs COUNT(columna)
    - 9 ejemplos prácticos
4. **RIGHT JOIN y FULL OUTER JOIN**
    - RIGHT JOIN como espejo de LEFT JOIN
    - Simulación de FULL OUTER JOIN en MySQL con UNION
    - Cuándo usar cada tipo
    - 4 ejemplos comparativos
5. **JOINs Especiales (CROSS JOIN y SELF JOIN)**
    - CROSS JOIN: producto cartesiano y casos de uso
    - SELF JOIN: comparaciones dentro de la misma tabla
    - Relaciones jerárquicas con SELF JOIN
    - JOINs con condiciones compuestas
    - 9 ejemplos avanzados
6. **JOINs Avanzados con Agregaciones**
    - JOINs combinados con COUNT, AVG, SUM
    - JOINs con subconsultas en FROM
    - Consultas de múltiples tablas (7+ tablas)
    - Manejo de NULL en agregaciones

- 8 ejemplos complejos

7. **JOIN vs Subconsultas**
    - Cuándo usar JOIN vs cuándo usar subconsultas
    - Comparación de rendimiento
    - NOT EXISTS vs LEFT JOIN + IS NULL
    - Tabla de decisión técnica
    - 4 ejemplos comparativos
8. **Errores Comunes y Soluciones**
    - Producto cartesiano accidental
    - Confusión ON vs WHERE en LEFT JOIN
    - Manejo incorrecto de NULL
    - Columnas ambiguas
    - Problemas de rendimiento
    - 4 ejemplos de diagnóstico

Parte Práctica (Secciones 9-12)

9. **Buenas Prácticas y Tabla de Decisión**

- Checklist completo (antes, durante, después)
- Diagrama de decisión: ¿qué JOIN usar?
- Recomendaciones de estilo y legibilidad
- Orden de ejecución de consultas
- Trucos de optimización

10. **Ejercicios Prácticos Clasificados**

- Nivel Básico (10 ejercicios) - INNER JOIN simple
- Nivel Intermedio (10 ejercicios) - LEFT/RIGHT JOIN
- Nivel Avanzado (8 ejercicios) - Múltiples JOINs + agregaciones
- Nivel Experto (5 ejercicios) - Casos complejos reales
- Todas las soluciones incluidas

11. **Casos Prácticos de Negocio**

- Sistema de recomendaciones personalizado
- Análisis de morosidad y recuperación de deuda
- Dashboard ejecutivo mensual
- Detección de fraude y anomalías
- Optimización de compras (qué libros adquirir)

12. **Script MySQL Completo y Verificado**

- Script de creación de base de datos (7 tablas)
- Carga completa de datos de ejemplo
- Tests automáticos de verificación

• Checklist final para el profesor

Contenido Técnico

Base de Datos de Ejemplo

El manual utiliza **BIBLIOTECA_DIGITAL**, una base de datos con 7 tablas relacionadas: **Especial**

Consultas. 3

Base de datos a Datos: Especial Join

Tabla Descripción Relaciones

AUTORES Escritores de libros 1:N con LIBROS
EDITORIALES Empresas editoras 1:N con LIBROS
LIBROS Catálogo de libros N:1 con AUTORES, EDITORIALES
USUARIOS Socios de la biblioteca 1:N con PRESTAMOS, VALORACIONES
PRESTAMOS Registro de préstamos N:1 con USUARIOS, LIBROS
VALORACIONES Puntuaciones de libros N:1 con USUARIOS, LIBROS
MULTAS Sanciones por retraso N:1 con PRESTAMOS

Estadísticas del Manual

- **47 ejemplos SQL** numerados y verificados
- **33 ejercicios prácticos** con soluciones completas
- **5 casos de negocio** reales y detallados
- **Script completo** ejecutable sin errores
- **Tests automáticos** de verificación incluidos

Tabla de contenido:

• Especial Consultas.

– *Join*

Duración: 8-10 horas lectivas
Resultados de Aprendizaje:

• RA3 - Criterio 3c: Consultas sobre varias tablas mediante composiciones internas (10%) • RA3 - Criterio 3d: Consultas sobre varias tablas mediante composiciones externas (15%)

Esta unidad fue desarrollada siguiendo los contenidos establecidos en la programación didáctica del módulo de Base de Datos para el ciclo formativo de Técnico Superior en Desarrollo de Aplicaciones Multiplataforma, adaptándose a los requerimientos del BOE y las directrices autonómicas vigentes.

Desarrollado por David Valbuena Segura con fecha Diciembre 2025.
Versión: 1.0

4 Especial Consultas.

CAPÍTULO 1

MANUAL DIDÁCTICO: JOINS EN MySQL

Módulo 0484 - Bases de Datos | 1º DAM
Resultado de Aprendizaje: RA3 (Criterios 3c, 3d)
Duración: 8-10 horas lectivas

1.1 SECCIÓN 1: FUNDAMENTOS DE JOINS

1.1.1 1.1. ¿Qué es un JOIN y por qué lo necesitamos?

Hasta ahora has trabajado con consultas sobre **una sola tabla**:

```
SELECT * FROM productos WHERE precio > 50;
```

Pero en bases de datos relacionales, **la información está distribuida en múltiples tablas** para evitar

redundancia. Por ejemplo:

❌ **MAL DISEÑO** (todo en una tabla):

PRESTAMOS_TODO
---
id_prestamo | nombre_usuario | email_usuario | titulo_libro | autor_libro | isbn | ...

Problema: Si un usuario hace 10 préstamos, sus datos se repiten 10 veces.

✅ **BUEN DISEÑO** (tablas separadas):

USUARIOS PRESTAMOS LIBROS
--- --- ---
id_usuario id_prestamo id_libro
nombre id_usuario (FK) titulo
email id_libro (FK) isbn
fecha_prestamo autor

¿El problema? Si quieres saber **«qué libros ha prestado cada usuario»**, necesitas **combinar información de 3 tablas**. **La solución: JOINs** → Une tablas relacionadas a través de claves foráneas.

1.1.2 1.2. Concepto de clave foránea y relaciones entre tablas

Clave Primaria (PK): Identifica únicamente cada fila de una tabla.
Clave Foránea (FK): Columna que **referencia** la PK de otra tabla.

Ejemplo:

USUARIOS PRESTAMOS
--- ---
id_usuario (PK) id_prestamo (PK)
nombre id_usuario (FK) ──► referencia a USUARIOS.id_usuario email id_libro (FK) ──► referencia a LIBROS.id_libro

La FK **conecta** las tablas → es el «puente» que usamos en los JOINs.

1.1.3 1.3. Tipos de JOINs: panorama general

Tipo de JOIN ¿Qué devuelve? Uso típico

INNER JOIN Solo filas con coincidencias en **ambas** tablas «Usuarios **que tienen** préstamos»
LEFT JOIN Todas las filas de la tabla **izquierda** + «Usuarios **aunque no tengan** présta mos»
coinciden cias
RIGHT JOIN Todas las filas de la tabla **derecha** + coincidencias Menos común (equivale a LEFT
inverti do)
FULL OUTER JOIN soporta nativamente
Todas las filas de **ambas** tablas MySQL no lo

CROSS JOIN Producto cartesiano (todas vs todas) Combinaciones exhaustivas **SELF**
JOIN Una tabla consigo misma «Libros del mismo autor»

No te preocupes si no lo entiendes aún → lo veremos paso a paso con ejemplos.

1.1.4 1.4. Base de datos a utilizar en este manual

Vamos a trabajar con una **BIBLIOTECA DIGITAL** que gestiona:

- 2/7 Libros y autores
- 2/7 Usuarios que prestan libros
- 2/7 Préstamos activos y devueltos
- ⭐ Valoraciones de libros
- 2/7 Editoriales que publican libros

Esquema de tablas (simplificado):

```
AUTORES
- id_autor (PK)
- nombre_autor
- nacionalidad

LIBROS
- id_libro (PK)
- titulo
- id_autor (FK → AUTORES)
- id_editorial (FK → EDITORIALES)
- anio_publicacion
- isbn

EDITORIALES
- id_editorial (PK)
- nombre_editorial
- pais

USUARIOS
- id_usuario (PK)
- nombre_usuario
- email
- fecha_registro

PRESTAMOS
- id_prestamo (PK)
- id_usuario (FK → USUARIOS)
- id_libro (FK → LIBROS)
- fecha_prestamo
- fecha_devolucion
- devuelto (BOOLEAN)

VALORACIONES
- id_valoracion (PK)
- id_usuario (FK → USUARIOS)
- id_libro (FK → LIBROS)
- puntuacion (1-5)
- comentario
```

- fecha_valoracion

MULTAS
- id_multa (PK)
- id_prestamo (FK → PRESTAMOS)
- importe
- pagada (BOOLEAN)

Base de datos a Datos: Especial Join

Diagrama de relaciones:

```
AUTORES (1) ──────── (N) LIBROS (N) ──────── (1) EDITORIALES
                          │
                          │ (1)
                          │
                     (N) PRESTAMOS (N) ──────── (1) USUARIOS
                          │ │
                          │ (1) │ (1)
                          │ │
                     (N) MULTAS (N) VALORACIONES (N)
                                                  │
                                                  └──────── (1) LIBROS
```

Esta BD nos permitirá practicar TODOS los tipos de JOIN.

1.2 SECCIÓN 2: INNER JOIN - COMPOSICIONES INTERNAS

1.2.1 2.1. Concepto de INNER JOIN

INNER JOIN devuelve solo las filas que tienen coincidencias en AMBAS tablas.

Analogía visual:

```
Tabla A Tabla B INNER JOIN
------- ------- ------------
   A B A ∩ B
   ┌───┐ ┌───┐ ┌─────┐
   │ 1│ │ 2│ │ │
   │ 2│ ←─────────→ │ 3│ ──────────► │2,3│ (solo coincidencias)
   │ 3│ │ 4│ │ │
   │ 5│ └───┘ └─────┘
   └───┘
```

Caso real:

- Tabla LIBROS tiene libros (algunos con autor, otros sin autor asignado aún)
- Tabla AUTORES tiene autores (algunos con libros publicados, otros sin)
- **INNER JOIN** → solo muestra libros **que tienen autor asignado**

1.2.2 2.2. Sintaxis básica de INNER JOIN

```
SELECT columnas
FROM tabla1
INNER JOIN tabla2 ON tabla1.columna_FK = tabla2.columna_PK;
```

Elementos clave:

- INNER JOIN → tipo de unión
- ON → condición de unión (¿qué columnas conectan las tablas?)
- tabla1.columna_FK = tabla2.columna_PK → igualamos FK con PK

Nota: INNER JOIN y JOIN son **idénticos** (INNER es opcional).

1.2.3 2.3. INNER JOIN con dos tablas - Ejemplos básicos

Ejemplo 1: Libros con su autor

Objetivo: Mostrar el título de cada libro junto al nombre de su autor.

```
SELECT libros.titulo, autores.nombre_autor
FROM libros
INNER JOIN autores ON libros.id_autor = autores.id_autor;
```

Resultado esperado:

titulo nombre_autor
Cien años de soledad Gabriel García Márquez
El principito Antoine de Saint-Exupéry
1984 George Orwell
Don Quijote de la Mancha Miguel de Cervantes
Crimen y castigo Fiódor Dostoyevski
… …

Explicación:

- Une LIBROS y AUTORES
- La condición libros.id_autor = autores.id_autor conecta las tablas
- Solo muestra libros **que tienen un autor asignado**

Ejemplo 2: Préstamos con nombre de usuario

Objetivo: Ver qué usuario ha realizado cada préstamo.

```
SELECT prestamos.id_prestamo,
       usuarios.nombre_usuario,
       prestamos.fecha_prestamo
FROM prestamos
INNER JOIN usuarios ON prestamos.id_usuario = usuarios.id_usuario;
```

Resultado esperado:

id_prestamo nombre_usuario fecha_prestamo

1 Ana García 2024-11-15
2 Carlos López 2024-11-18
3 Ana García 2024-11-20
4 María Fernández 2024-11-22
5 Pedro Sánchez 2024-11-25
… … …

Explicación:

- Une PRESTAMOS y USUARIOS
- Solo muestra préstamos de usuarios registrados (todos deberían estarlo)

1.2.4 2.4. Alias de tablas y nombres cualificados

Problema: Escribir libros.titulo y autores.nombre_autor es tedioso.

Solución: Usa **alias** (abreviaturas) para las tablas.

Ejemplo 3: Mismo que Ejemplo 1, con alias

```
SELECT l.titulo, a.nombre_autor
FROM libros AS l
INNER JOIN autores AS a ON l.id_autor = a.id_autor;
```

Resultado: Idéntico al Ejemplo 1.

Ventajas:

- Código más corto y legible
- AS es opcional → FROM libros l también funciona
- **Convención:** usa iniciales o abreviaturas claras (l = libros, a = autores)

2/7 Importante: Si usas alias, **debes usarlo en toda la consulta**:

```
-- ERROR: no puedes mezclar nombre real y alias
SELECT libros.titulo, a.nombre_autor
FROM libros AS l
INNER JOIN autores AS a ON l.id_autor = a.id_autor;

-- CORRECTO
SELECT l.titulo, a.nombre_autor
FROM libros AS l
INNER JOIN autores AS a ON l.id_autor = a.id_autor;
```

Ejemplo 4: Libros con su editorial

```
SELECT l.titulo, e.nombre_editorial, l.anio_publicacion
FROM libros l
INNER JOIN editoriales e ON l.id_editorial = e.id_editorial;
```

Resultado esperado:

titulo nombre_editorial anio_publicacion
Cien años de soledad Editorial Sudamericana 1967

El principito Gallimard 1943
1984 Secker & Warburg 1949
Don Quijote de la Mancha Francisco de Robles 1605
… … …

1.2.5 2.5. Sintaxis antigua (WHERE) vs moderna (JOIN)

Antes de SQL-92, los JOINs se escribían con WHERE:

Sintaxis antigua (fi NO recomendada):

```
SELECT l.titulo, a.nombre_autor
FROM libros l, autores a
WHERE l.id_autor = a.id_autor;
```

Sintaxis moderna (fi RECOMENDADA):

```
SELECT l.titulo, a.nombre_autor
FROM libros l
INNER JOIN autores a ON l.id_autor = a.id_autor;
```

¿Por qué es mejor JOIN?

1. **Claridad:** Separa la condición de unión (ON) del filtrado (WHERE)
2. **Previene errores:** Evita productos cartesianos accidentales
3. **Estándar:** Es el estándar SQL moderno (SQL-92 en adelante)
4. **Obligatorio para OUTER JOINs:** LEFT/RIGHT JOIN **requieren** sintaxis moderna

Regla: Usa **siempre** la sintaxis moderna con JOIN ... ON.

1.2.6 2.6. INNER JOIN con tres o más tablas

Puedes encadenar múltiples JOINs para unir 3, 4, 5… tablas.

Ejemplo 5: Préstamos con usuario y libro

Objetivo: Ver quién ha prestado qué libro.

```
SELECT u.nombre_usuario,
       l.titulo,
       p.fecha_prestamo
FROM prestamos p
INNER JOIN usuarios u ON p.id_usuario = u.id_usuario
INNER JOIN libros l ON p.id_libro = l.id_libro;
```

Resultado esperado:

nombre_usuario titulo fecha_prestamo

Ana García 1984 2024-11-15
Carlos López El principito 2024-11-18
Ana García Cien años de soledad 2024-11-20
María Fernández Don Quijote de la Mancha 2024-11-22
Pedro Sánchez Crimen y castigo 2024-11-25
… … …

Explicación:

1. Empieza en PRESTAMOS (tabla «central»)
2. Une con USUARIOS para obtener el nombre del usuario
3. Une con LIBROS para obtener el título del libro

Ejemplo 6: Préstamos completos (usuario, libro, autor, editorial)

```
SELECT u.nombre_usuario,
       l.titulo,
       a.nombre_autor,
       e.nombre_editorial,
       p.fecha_prestamo
FROM prestamos p
INNER JOIN usuarios u ON p.id_usuario = u.id_usuario
INNER JOIN libros l ON p.id_libro = l.id_libro
INNER JOIN autores a ON l.id_autor = a.id_autor
INNER JOIN editoriales e ON l.id_editorial = e.id_editorial;
```

Resultado esperado:

nombre_usuario titulo nombre_autor nombre_editorial fecha_prestamo

Ana García 1984 George Orwell Secker & Warburg 2024-11-15 Carlos López El principito Antoine de Saint-Exupéry Gallimard 2024-11-18 … … … … …

Explicación:

- Une **5 tablas** en cadena
- El orden importa: empieza en PRESTAMOS y ve «saltando» por las FKs

Ejemplo 7: Valoraciones con usuario y libro

```
SELECT u.nombre_usuario,
       l.titulo,
       v.puntuacion,
       v.comentario
FROM valoraciones v
INNER JOIN usuarios u ON v.id_usuario = u.id_usuario
INNER JOIN libros l ON v.id_libro = l.id_libro;
```

Resultado esperado:

nombre_usuario titulo puntuacion comentario

Ana García 1984 5 Obra maestra distópica
Carlos López El principito 4 Hermoso y filosófico
María Fernández Don Quijote 5 Clásico imprescindible
Pedro Sánchez Crimen y castigo 5 Profunda exploración psicológica
… … … …

1.2.7 2.7. Condiciones ON vs WHERE: diferencias críticas

ON → Condición de **unión** (¿cómo se conectan las tablas?)
WHERE → Condición de **filtrado** (¿qué filas quiero del resultado?)

Ejemplo 8: Préstamos de libros publicados después de 2000

```
SELECT u.nombre_usuario,
       l.titulo,
       l.anio_publicacion,
       p.fecha_prestamo
FROM prestamos p
INNER JOIN usuarios u ON p.id_usuario = u.id_usuario
INNER JOIN libros l ON p.id_libro = l.id_libro
WHERE l.anio_publicacion > 2000;
```

Resultado esperado:

nombre_usuario titulo anio_publicacion fecha_prestamo
Ana García El código Da Vinci 2003 2024-11-15
Carlos López Los juegos del hambre 2008 2024-11-20
… … … …

Explicación:

- ON → une las tablas
- WHERE → filtra solo libros con anio_publicacion > 2000

Ejemplo 9: Préstamos aún no devueltos

```
SELECT u.nombre_usuario,
       l.titulo,
       p.fecha_prestamo,
       p.fecha_devolucion
FROM prestamos p
INNER JOIN usuarios u ON p.id_usuario = u.id_usuario
INNER JOIN libros l ON p.id_libro = l.id_libro
WHERE p.devuelto = FALSE;
```

Resultado esperado:

nombre_usuario titulo fecha_prestamo fecha_devolucion
Ana García Cien años de soledad 2024-11-20 NULL
Pedro Sánchez Crimen y castigo 2024-11-25 NULL
… … … …

fi ERROR COMÚN: Poner filtros en ON en vez de WHERE

```
-- MAL (funciona pero es confuso)
SELECT u.nombre_usuario, l.titulo
FROM prestamos p
INNER JOIN usuarios u ON p.id_usuario = u.id_usuario
INNER JOIN libros l ON p.id_libro = l.id_libro AND l.anio_publicacion > 2000;

-- BIEN (claro y correcto)
SELECT u.nombre_usuario, l.titulo
FROM prestamos p
INNER JOIN usuarios u ON p.id_usuario = u.id_usuario
INNER JOIN libros l ON p.id_libro = l.id_libro
WHERE l.anio_publicacion > 2000;
```

Regla:

• ON → **solo** para igualar claves (FK = PK)

• WHERE → para **cualquier** otro filtro

Excepción: En LEFT JOIN la diferencia **sí importa** (lo veremos en Sección 3).

1.3 SECCIÓN 5: JOINS ESPECIALES

1.3.1 5.1. CROSS JOIN (producto cartesiano)

CROSS JOIN combina CADA fila de la tabla A con CADA fila de la tabla B.

Analogía visual:

```
Tabla A Tabla B CROSS JOIN
------- ------- -----------
A B Todas las combinaciones posibles
┌───┐ ┌───┐ ┌─────┐
│ 1│ │ X│ │1-X │
│ 2│ × │ Y│ ──────────► │1-Y │
└───┘ └───┘ │2-X │
│2-Y │
└─────┘
```

Fórmula: Si A tiene n filas y B tiene m filas → CROSS JOIN devuelve n × m filas.

2/7 Cuidado: CROSS JOIN genera **muchísimas filas** → úsalo solo cuando realmente lo necesites. **14**

Capítulo 1. MANUAL DIDÁCTICO: JOINS EN MySQL

Base de datos a Datos: Especial Join

Ejemplo 23: Producto cartesiano simple - Usuarios × Libros

```
SELECT u.nombre_usuario, l.titulo
FROM usuarios u
CROSS JOIN libros l
LIMIT 10;
```

Resultado esperado (primeras 10 filas):

nombre_usuario	titulo
Ana García	1984
Ana García	El principito
Ana García	Cien años de soledad
Ana García	Don Quijote de la Mancha
Ana García	Crimen y castigo
Ana García	La metamorfosis
Ana García	El amor en los tiempos del cólera
Carlos López	1984
Carlos López	El principito
Carlos López	Cien años de soledad

Explicación:

- Si tenemos 7 usuarios y 10 libros → 7 × 10 = **70 filas**
- Cada usuario aparece con cada libro
- **No tiene sentido práctico** en este caso (solo para demostración)

Ejemplo 24: Caso práctico - Combinaciones de turnos y salas

Escenario: La biblioteca tiene 3 salas de lectura y 4 turnos horarios. Queremos generar todas las posibles combinaciones para el planning.

```
-- Creamos tablas temporales para el ejemplo
CREATE TEMPORARY TABLE salas (
    id_sala INT,
    nombre_sala VARCHAR(50)
);

CREATE TEMPORARY TABLE turnos (
    id_turno INT,
    horario VARCHAR(20)
);

INSERT INTO salas VALUES
 (1, 'Sala Cervantes'),
 (2, 'Sala Borges'),
 (3, 'Sala García Márquez');

INSERT INTO turnos VALUES
 (1, '09:00-11:00'),
```

(continúe en la próxima página)

(proviene de la página anterior)

```
 (2, '11:00-13:00'),
 (3, '15:00-17:00'),
 (4, '17:00-19:00');

-- CROSS JOIN para generar planning completo
SELECT s.nombre_sala, t.horario
FROM salas s
CROSS JOIN turnos t
ORDER BY s.nombre_sala, t.id_turno;
```

Resultado esperado:

nombre_sala	horario
Sala Borges	09:00-11:00
Sala Borges	11:00-13:00
Sala Borges	15:00-17:00
Sala Borges	17:00-19:00
Sala Cervantes	09:00-11:00
Sala Cervantes	11:00-13:00
Sala Cervantes	15:00-17:00
Sala Cervantes	17:00-19:00
Sala García Márquez	09:00-11:00
Sala García Márquez	11:00-13:00
Sala García Márquez	15:00-17:00
Sala García Márquez	17:00-19:00

Explicación:

- 3 salas × 4 turnos = **12 combinaciones**

- Útil para: generar calendarios, planificaciones, combinaciones de productos

Ejemplo 25: CROSS JOIN implícito (sintaxis antigua)

```
-- Sintaxis moderna (explícita)
SELECT u.nombre_usuario, l.titulo
FROM usuarios u
CROSS JOIN libros l;

-- Sintaxis antigua (implícita) - SIN condición ON/WHERE
SELECT u.nombre_usuario, l.titulo
FROM usuarios u, libros l;
```

Ambas consultas son idénticas.

2/7 Peligro: Si olvidas el WHERE en la sintaxis antigua, obtienes un producto cartesiano accidental.

```
--  ERROR COMÚN: olvidar WHERE → producto cartesiano
SELECT u.nombre_usuario, l.titulo
FROM usuarios u, libros l; -- ¡7 × 10 = 70 filas sin sentido!

--  CORRECTO: con WHERE actúa como INNER JOIN
SELECT u.nombre_usuario, l.titulo
```

(continúe en la próxima página)

(proviene de la página anterior)

```
FROM usuarios u, libros l
WHERE u.id_usuario = l.id_libro; -- esto no tiene sentido pero evita cartesiano
```

Por eso recomendamos: Usa siempre JOIN ... ON explícito.

1.3.2 5.2. SELF JOIN (auto-uniones)

SELF JOIN une una tabla consigo misma → útil para relaciones jerárquicas o comparaciones dentro de la misma tabla.

Casos de uso:

- Empleados y sus supervisores (ambos en tabla EMPLEADOS)
- Productos relacionados (ambos en tabla PRODUCTOS)
- Usuarios que prestaron los mismos libros

Ejemplo 26: Libros del mismo autor

Objetivo: Encontrar pares de libros escritos por el mismo autor.

```
SELECT l1.titulo AS libro1,
       l2.titulo AS libro2,
       a.nombre_autor
FROM libros l1
INNER JOIN libros l2 ON l1.id_autor = l2.id_autor AND l1.id_libro < l2.id_libro INNER JOIN autores a ON
l1.id_autor = a.id_autor
ORDER BY a.nombre_autor, l1.titulo;
```

Resultado esperado:

libro1 libro2 nombre_autor

Cien años de soledad El amor en los tiempos del cólera Gabriel García Márquez Cien años de soledad El otoño del patriarca Gabriel García Márquez El amor en los tiempos del cólera El otoño del patriarca Gabriel García Márquez 1984 Rebelión en la granja George Orwell

Explicación:

- l1 y l2 son alias de la **misma** tabla LIBROS
- l1.id_autor = l2.id_autor → mismo autor
- l1.id_libro < l2.id_libro → evita duplicados (libro1-libro2 y libro2-libro1) •

También evita comparar un libro consigo mismo

Sin la condición <:

```
--  Sin filtro: aparecen duplicados y auto-comparaciones
SELECT l1.titulo, l2.titulo
FROM libros l1
JOIN libros l2 ON l1.id_autor = l2.id_autor;
```

(continúe en la próxima página)

Base de datos a Datos: Especial Join

(proviene de la página anterior)

```
-- Resultado incluiría:
-- | 1984 | 1984 | (libro consigo mismo)
-- | 1984 | Rebelión en la granja |
-- | Rebelión en la granja | 1984 | (duplicado)
```

Ejemplo 27: Usuarios que leyeron el mismo libro

```
SELECT DISTINCT u1.nombre_usuario AS usuario1,
        u2.nombre_usuario AS usuario2,
        l.titulo AS libro_en_comun
FROM prestamos p1
INNER JOIN prestamos p2 ON p1.id_libro = p2.id_libro AND p1.id_usuario < p2.id_usuario INNER JOIN usuarios
u1 ON p1.id_usuario = u1.id_usuario
INNER JOIN usuarios u2 ON p2.id_usuario = u2.id_usuario
INNER JOIN libros l ON p1.id_libro = l.id_libro
ORDER BY l.titulo, u1.nombre_usuario;
```

Resultado esperado:

usuario1 usuario2 libro_en_comun

Ana García Carlos López 1984
Ana García Pedro Sánchez 1984
Carlos López Pedro Sánchez 1984
Ana García María Fernández Cien años de soledad

Explicación:

- p1 y p2 son alias de PRESTAMOS
- p1.id_libro = p2.id_libro → mismo libro
- p1.id_usuario < p2.id_usuario → evita duplicados

- DISTINCT → elimina repeticiones si ambos lo prestaron varias veces
- **Uso práctico:** Sistema de recomendaciones («usuarios con gustos similares»)

Ejemplo 28: Estructura jerárquica simulada - Secciones y subsecciones de biblioteca

Escenario: La biblioteca tiene secciones (ej: «Literatura») y subsecciones (ej: «Literatura española», «Literatura latino americana») en la misma tabla.

```
-- Tabla con relación jerárquica (un campo apunta a su propio ID)
CREATE TEMPORARY TABLE secciones_biblioteca (
    id_seccion INT PRIMARY KEY,
    nombre_seccion VARCHAR(100),
    id_seccion_padre INT -- apunta a secciones_biblioteca.id_seccion
);

INSERT INTO secciones_biblioteca VALUES
 (1, 'Literatura', NULL),
 (2, 'Literatura española', 1),
 (3, 'Literatura latinoamericana', 1),
```

(continúe en la próxima página)

(proviene de la página anterior)

```
 (4, 'Ciencia', NULL),
 (5, 'Física', 4),
 (6, 'Química', 4),
 (7, 'Historia', NULL);

-- SELF JOIN: mostrar secciones padre e hijas
SELECT padre.nombre_seccion AS seccion_padre,
       hija.nombre_seccion AS subseccion
FROM secciones_biblioteca padre
INNER JOIN secciones_biblioteca hija ON padre.id_seccion = hija.id_seccion_padre ORDER BY
padre.nombre_seccion, hija.nombre_seccion;
```

Resultado esperado:

seccion_padre	subseccion
Ciencia	Física
Ciencia	Química
Literatura	Literatura española
Literatura	Literatura latinoamericana

Explicación:

- padre y hija son alias de la misma tabla
- hija.id_seccion_padre = padre.id_seccion → conecta hijo con padre
- «Historia» no aparece (no tiene subsecciones)

1.3.3 5.3. JOIN con condiciones compuestas (múltiples columnas)

A veces la relación entre tablas requiere más de una columna.

Ejemplo 29: Tabla de eventos y asistencias con doble clave

```sql
-- Escenario: eventos de biblioteca (charlas, talleres) y asistencias CREATE TEMPORARY
TABLE eventos (
    fecha_evento DATE,
    hora_evento TIME,
    nombre_evento VARCHAR(100),
    PRIMARY KEY (fecha_evento, hora_evento)
);

CREATE TEMPORARY TABLE asistencias (
    fecha_evento DATE,
    hora_evento TIME,
    id_usuario INT,
    asistio BOOLEAN
);

INSERT INTO eventos VALUES
 ('2024-12-01', '10:00:00', 'Taller de escritura creativa'),
 ('2024-12-01', '16:00:00', 'Charla: García Márquez'),
```

(continúe en la próxima página)

Base de datos a Datos: Especial Join

(proviene de la página anterior)

```sql
 ('2024-12-05', '10:00:00', 'Club de lectura');

INSERT INTO asistencias VALUES
 ('2024-12-01', '10:00:00', 1, TRUE),
 ('2024-12-01', '10:00:00', 2, TRUE),
 ('2024-12-01', '16:00:00', 1, FALSE),
 ('2024-12-05', '10:00:00', 3, TRUE);

-- JOIN con condición compuesta (2 columnas)
SELECT e.nombre_evento,
       u.nombre_usuario,
       a.asistio
FROM asistencias a
INNER JOIN eventos e ON a.fecha_evento = e.fecha_evento
                    AND a.hora_evento = e.hora_evento
INNER JOIN usuarios u ON a.id_usuario = u.id_usuario;
```

Resultado esperado:

nombre_evento	nombre_usuario	asistio
Taller de escritura creativa	Ana García	TRUE
Taller de escritura creativa	Carlos López	TRUE
Charla: García Márquez	Ana García	FALSE
Club de lectura	María Fernández	TRUE

Explicación:

- La PK de EVENTOS es compuesta: (fecha_evento, hora_evento)
- El JOIN debe igualar **ambas** columnas
- AND dentro del ON para condiciones múltiples

1.3.4 5.4. JOIN con funciones y expresiones

Puedes usar funciones en las condiciones de JOIN.

Ejemplo 30: Usuarios que se registraron el mismo mes

```
SELECT u1.nombre_usuario AS usuario1,
       u2.nombre_usuario AS usuario2,
       DATE_FORMAT(u1.fecha_registro, '%Y-%m') AS mes_registro
FROM usuarios u1
INNER JOIN usuarios u2 ON YEAR(u1.fecha_registro) = YEAR(u2.fecha_registro) AND
                          MONTH(u1.fecha_registro) = MONTH(u2.fecha_registro)
                          AND u1.id_usuario < u2.id_usuario
ORDER BY mes_registro, u1.nombre_usuario;
```

Resultado esperado:

usuario1 usuario2 mes_registro

Ana García Carlos López 2024-03
David Ruiz Elena Moreno 2024-08

Explicación:

- YEAR() y MONTH() extraen año y mes
- Encuentra usuarios registrados en el mismo mes/año
- **Uso práctico:** Análisis de cohortes de usuarios

Ejemplo 31: Préstamos con rango de fechas superpuestas

```
-- Préstamos que se superponen en el tiempo (mismo libro prestado simultáneamente -␣ ↪ERROR de gestión)
SELECT p1.id_prestamo AS prestamo1,
       p2.id_prestamo AS prestamo2,
       l.titulo,
       p1.fecha_prestamo AS inicio1,
       p1.fecha_devolucion AS fin1,
       p2.fecha_prestamo AS inicio2,
       p2.fecha_devolucion AS fin2
FROM prestamos p1
INNER JOIN prestamos p2 ON p1.id_libro = p2.id_libro
                          AND p1.id_prestamo < p2.id_prestamo
                            AND p1.fecha_prestamo <= p2.fecha_devolucion
                            AND p2.fecha_prestamo <= p1.fecha_devolucion
INNER JOIN libros l ON p1.id_libro = l.id_libro
WHERE p1.fecha_devolucion IS NOT NULL
  AND p2.fecha_devolucion IS NOT NULL;
```

Resultado esperado:

prestamo1 prestamo2 titulo inicio1 fin1 inicio2 fin2

12 15 1984 2024-11-10 2024-11-25 2024-11-20 2024-12-05

Explicación:

- Detecta préstamos **superpuestos** del mismo libro (error de gestión)
- Condiciones de superposición:

  – p1.fecha_prestamo <= p2.fecha_devolucion → p1 empieza antes de que termine p2 **–** p2.fecha_prestamo <= p1.fecha_devolucion → p2 empieza antes de que termine p1 • **Uso práctico:** Auditoría de integridad de datos

Base de datos a Datos: Especial Join

1.4 SECCIÓN 6: JOINS AVANZADOS

1.4.1 6.1. JOINs con agregaciones (GROUP BY + JOIN)

Combinar JOINs con funciones agregadas es muy común en informes.

Ejemplo 32: Número de préstamos por usuario

```
SELECT u.nombre_usuario,
        COUNT(p.id_prestamo) AS total_prestamos,
        COUNT(CASE WHEN p.devuelto = FALSE THEN 1 END) AS prestamos_activos FROM
usuarios u
LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario
GROUP BY u.id_usuario, u.nombre_usuario
ORDER BY total_prestamos DESC;
```

Resultado esperado:

nombre_usuario	total_prestamos	prestamos_activos
Ana García	5	1
Pedro Sánchez	3	1
Carlos López	2	0
María Fernández	1	0
David Ruiz	0	0
Elena Moreno	0	0

Explicación:

- LEFT JOIN → incluye usuarios sin préstamos
- COUNT(p.id_prestamo) → cuenta solo préstamos (no cuenta NULL)
- COUNT(CASE WHEN ...) → cuenta condicional (solo activos)

Ejemplo 33: Libros más populares (por número de préstamos)

```
SELECT l.titulo,
```

```
        a.nombre_autor,
        COUNT(p.id_prestamo) AS veces_prestado,
        MIN(p.fecha_prestamo) AS primer_prestamo,
        MAX(p.fecha_prestamo) AS ultimo_prestamo
FROM libros l
INNER JOIN autores a ON l.id_autor = a.id_autor
LEFT JOIN prestamos p ON l.id_libro = p.id_libro
GROUP BY l.id_libro, l.titulo, a.nombre_autor
HAVING COUNT(p.id_prestamo) > 0
ORDER BY veces_prestado DESC
LIMIT 5;
```


Resultado esperado:

titulo nombre_autor veces_prestado primer_prestamo ultimo_prestamo

1984 George Orwell 5 2024-09-15 2024-11-28 El principito Antoine de Saint-Exupéry 4 2024-10-01 2024-11-25 Cien años de soledad Gabriel García Márquez 3 2024-09-20 2024-11-20 Don Quijote Miguel de Cervantes 2 2024-10-15 2024-11-18 Crimen y castigo Fiódor Dostoyevski 1 2024-11-25 2024-11-25

Explicación:

- HAVING COUNT(...) > 0 → filtra libros sin préstamos
- MIN/MAX → primera y última vez prestado
- **Uso práctico:** Dashboard de popularidad

Ejemplo 34: Autores más valorados (promedio de puntuaciones)

```
SELECT a.nombre_autor,
        COUNT(DISTINCT l.id_libro) AS libros_publicados,
        COUNT(v.id_valoracion) AS num_valoraciones,
        ROUND(AVG(v.puntuacion), 2) AS puntuacion_media,
        MIN(v.puntuacion) AS peor_valoracion,
        MAX(v.puntuacion) AS mejor_valoracion
FROM autores a
LEFT JOIN libros l ON a.id_autor = l.id_autor
LEFT JOIN valoraciones v ON l.id_libro = v.id_libro
GROUP BY a.id_autor, a.nombre_autor
HAVING num_valoraciones > 0
ORDER BY puntuacion_media DESC, num_valoraciones DESC;
```

Resultado esperado:

bros_publicados peor_valoracionme
num_valoracionespuntua jor_valoracion
nombre_autor li cion_media

Miguel de Cervantes 1 3 5.00 5 5 Fiódor Dostoyevski 1 2 5.00 5 5
Gabriel García Már quez 3 4 4.75 4 5

George Orwell 2 5 4.60 4 5
Antoine de Saint Exupéry

1 2 4.50 4 5

Explicación:

- Dos LEFT JOIN encadenados
- COUNT(DISTINCT l.id_libro) → evita contar libros duplicados
- HAVING num_valoraciones > 0 → solo autores con valoraciones

1.4.2 6.2. JOINs con subconsultas combinados

Puedes usar subconsultas en el FROM, WHERE o JOIN.

Ejemplo 35: Usuarios con más préstamos que el promedio

```
SELECT u.nombre_usuario,
       COUNT(p.id_prestamo) AS total_prestamos
FROM usuarios u
INNER JOIN prestamos p ON u.id_usuario = p.id_usuario
GROUP BY u.id_usuario, u.nombre_usuario
HAVING COUNT(p.id_prestamo) > (
    SELECT AVG(prestamos_por_usuario)
    FROM (
        SELECT COUNT(*) AS prestamos_por_usuario
        FROM prestamos
        GROUP BY id_usuario
    ) AS subconsulta
)
ORDER BY total_prestamos DESC;
```

Resultado esperado:

nombre_usuario	total_prestamos
Ana García	5
Pedro Sánchez	3

Explicación:

- Subconsulta calcula promedio de préstamos por usuario
- Solo muestra usuarios por encima del promedio
- **Promedio calculado:** (5+3+2+1) / 4 = 2.75 → muestra usuarios con ≥3

Ejemplo 36: Libros nunca prestados de autores populares

```
SELECT l.titulo,
       a.nombre_autor,
       l.anio_publicacion
FROM libros l
INNER JOIN autores a ON l.id_autor = a.id_autor
```

```
INNER JOIN (
    -- Autores con al menos 2 libros prestados
    SELECT DISTINCT l2.id_autor
    FROM prestamos p
    INNER JOIN libros l2 ON p.id_libro = l2.id_libro
    GROUP BY l2.id_autor
    HAVING COUNT(DISTINCT p.id_libro) >= 2
) AS autores_populares ON l.id_autor = autores_populares.id_autor
LEFT JOIN prestamos p ON l.id_libro = p.id_libro
WHERE p.id_prestamo IS NULL;
```


Resultado esperado:

titulo	nombre_autor	anio_publicacion
Rebelión en la granja	George Orwell	1945
El otoño del patriarca	Gabriel García Márquez	1975

Explicación:

- Subconsulta identifica autores con ≥2 libros prestados (autores «populares»)
- Luego busca libros de esos autores que nunca se prestaron
- **Uso práctico:** «Libros olvidados de autores conocidos» → candidatos para promoción

1.4.3 6.3. JOINs anidados complejos

Ejemplo 37: Informe completo de préstamos con todas las relaciones

```
SELECT u.nombre_usuario,
        l.titulo,
        a.nombre_autor,
        e.nombre_editorial,
        p.fecha_prestamo,
        p.fecha_devolucion,
        DATEDIFF(COALESCE(p.fecha_devolucion, CURDATE()), p.fecha_prestamo) AS dias_prestamo,
        m.importe AS multa,
        v.puntuacion AS valoracion_usuario
FROM prestamos p
INNER JOIN usuarios u ON p.id_usuario = u.id_usuario
INNER JOIN libros l ON p.id_libro = l.id_libro
INNER JOIN autores a ON l.id_autor = a.id_autor
INNER JOIN editoriales e ON l.id_editorial = e.id_editorial
LEFT JOIN multas m ON p.id_prestamo = m.id_prestamo
LEFT JOIN valoraciones v ON p.id_usuario = v.id_usuario AND p.id_libro = v.id_libro ORDER BY
p.fecha_prestamo DESC;
```

Resultado esperado:

Base de datos a Datos: Especial Join

titulo nom dias_prestam
nom o mul
fe valora
nom fe
bre_usuario cha_prestamo cion_usuario Sánchez yevski NULL 8 NULL5
bre_autor cha_devolucion Pedro Crimen y castigo Editorial Planeta
bre_editorial ta Fiódor Dosto 2024-11- 25
dez Miguel de Francisco de 2024-11- 22 8 NULL5
María Fernán Don Qui jote Cervantes Robles 2024-11- 30
Editorial Gallimard cía Warburg
Sudameri cana 2024-11- 18 2024-11- 15
Ana Gar cía 2024-11- 20 Carlos López 2024-11- 28 2024-11- 25
Cien años de NULL 13 5.00 El principi to 10 NULL4 **Explicación:** 10 NULL5
soledad NULL Antoine 1984 George Or
Gabriel Gar cía de Saint well
Márquez Exupéry Secker &
Ana Gar

- 4 INNER JOIN → garantiza que todo tenga autor, editorial, usuario, libro
- 2 LEFT JOIN → multas y valoraciones son opcionales
- COALESCE(p.fecha_devolucion, CURDATE()) → si no está devuelto, usa fecha actual
- Condición doble en valoraciones: AND p.id_usuario = v.id_usuario AND p.id_libro = v. id_libro

1.4.4 6.4. LEFT JOIN con COUNT y manejo de NULL

Problema común: ¿Cómo contar correctamente con LEFT JOIN?

Ejemplo 38: Comparación de técnicas de conteo

```
SELECT u.nombre_usuario,
        COUNT(*) AS filas_totales, -- cuenta TODAS las filas␣ ↪(incluye NULL)
        COUNT(p.id_prestamo) AS prestamos_reales, -- cuenta solo préstamos␣ ↪(excluye NULL)
        COUNT(DISTINCT p.id_libro) AS libros_diferentes,
        SUM(CASE WHEN p.devuelto = FALSE THEN 1 ELSE 0 END) AS activos
FROM usuarios u
LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario
GROUP BY u.id_usuario, u.nombre_usuario;
```

Resultado esperado:

nombre_usuario filas_totales prestamos_reales libros_diferentes activos

Ana García 5 5 4 1
Carlos López 2 2 2 0
María Fernández 1 1 1 0
Pedro Sánchez 3 3 2 1
David Ruiz 1 0 0 0
Elena Moreno 1 0 0 0

Explicación:

- David y Elena NO tienen préstamos:
  - COUNT(*) = 1 (cuenta la fila con NULL)
  - COUNT(p.id_prestamo) = 0 (no cuenta NULL)
- SUM(CASE WHEN ...) → suma condicional (cuenta activos)

Ejemplo 39: Libros con valoración promedio (incluyendo libros sin valoraciones)

```
SELECT l.titulo,
        COUNT(v.id_valoracion) AS num_valoraciones,
        ROUND(AVG(v.puntuacion), 2) AS puntuacion_media,
        CASE
            WHEN COUNT(v.id_valoracion) = 0 THEN 'Sin valorar'
            WHEN AVG(v.puntuacion) >= 4.5 THEN 'Excelente'
            WHEN AVG(v.puntuacion) >= 3.5 THEN 'Bueno'
            ELSE 'Regular'
        END AS categoria
FROM libros l
LEFT JOIN valoraciones v ON l.id_libro = v.id_libro
GROUP BY l.id_libro, l.titulo
ORDER BY num_valoraciones DESC, puntuacion_media DESC;
```

Resultado esperado:

titulo num_valoraciones puntuacion_media categoria

1984 5 4.60 Excelente Cien años de soledad 4 4.75 Excelente Don Quijote 3 5.00 Excelente El principito 2 4.50 Excelente Crimen y castigo 2 5.00 Excelente La metamorfosis 0 NULL Sin valorar

El amor en los tiempos del cólera 0 NULL Sin valorar

Explicación:

- LEFT JOIN → incluye libros sin valoraciones

- COUNT(v.id_valoracion) = 0 para libros sin valorar
- AVG(v.puntuacion) = NULL para libros sin valorar
- CASE → categorización basada en promedio

1.5 SECCIÓN 7: JOIN VS SUBCONSULTAS

1.5.1 7.1. Comparación técnica y de rendimiento

Pregunta clave: ¿Cuándo usar JOIN y cuándo usar subconsultas?

Respuesta corta: Ambos pueden resolver los mismos problemas, pero tienen diferencias en legibilidad, rendimiento y casos de uso óptimos.

Ejemplo 40: Mismo resultado, dos enfoques

Objetivo: Usuarios que han hecho al menos un préstamo.

Enfoque 1 - Con JOIN:

```
SELECT DISTINCT u.nombre_usuario, u.email
FROM usuarios u
INNER JOIN prestamos p ON u.id_usuario = p.id_usuario;
```

Enfoque 2 - Con subconsulta:

```
SELECT u.nombre_usuario, u.email
FROM usuarios u
WHERE u.id_usuario IN (
    SELECT DISTINCT id_usuario
    FROM prestamos
);
```

Resultado esperado (idéntico en ambos):

nombre_usuario	email
Ana García	ana.garcia@email.com
Carlos López	carlos.lopez@email.com
María Fernández	maria.fernandez@email.com
Pedro Sánchez	pedro.sanchez@email.com

Comparación:

Aspecto	JOIN	Subconsulta
Legibilidad	Más intuitivo para «combinar tablas»	Más intuitivo para «filtrar por condición»
Rendimien to	⚡ Generalmente más rápido 2/7	Puede ser más lento (depende del optimiza dor)
Duplicados	Requiere DISTINCT si hay múltiples coinciden cias	No requiere DISTINCT con IN

Columnas Permite seleccionar columnas de ambas tablas Solo de la tabla principal **28 Capítulo 1.**

MANUAL DIDÁCTICO: JOINS EN MySQL

Base de datos a Datos: Especial Join

Ejemplo 41: Libros del autor más prolífico

Enfoque 1 - Con JOIN:

```
SELECT l.titulo, a.nombre_autor, COUNT(*) OVER (PARTITION BY a.id_autor) AS libros_autor
FROM libros l
INNER JOIN autores a ON l.id_autor = a.id_autor
WHERE a.id_autor = (
    SELECT id_autor
    FROM libros
    GROUP BY id_autor
    ORDER BY COUNT(*) DESC
    LIMIT 1
);
```

Enfoque 2 - Con subconsulta en FROM:

```
SELECT l.titulo, a.nombre_autor, autor_stats.total_libros
FROM libros l
INNER JOIN autores a ON l.id_autor = a.id_autor
INNER JOIN (
    SELECT id_autor, COUNT(*) AS total_libros
    FROM libros
    GROUP BY id_autor
    ORDER BY COUNT(*) DESC
    LIMIT 1
) AS autor_stats ON l.id_autor = autor_stats.id_autor;
```

Resultado esperado:

titulo	nombre_autor	total_libros
Cien años de soledad	Gabriel García Márquez	3
El amor en los tiempos del cólera	Gabriel García Márquez	3
El otoño del patriarca	Gabriel García Márquez	3

Análisis:

- Ambos enfoques funcionan
- Subconsulta en FROM es más modular (puedes reutilizar autor_stats)

- JOIN + subconsulta escalar en WHERE es más compacto

1.5.2 7.2. ¿Cuándo usar JOIN?

✅ **Usa JOIN cuando:**

1. **Necesitas columnas de múltiples tablas**

```
-- JOIN: necesitas nombre_usuario Y titulo
SELECT u.nombre_usuario, l.titulo, p.fecha_prestamo
FROM prestamos p
INNER JOIN usuarios u ON p.id_usuario = u.id_usuario
INNER JOIN libros l ON p.id_libro = l.id_libro;
```


2. **Quieres todas las combinaciones (incluso sin coincidencias)**

```
-- LEFT JOIN: necesitas TODOS los usuarios (incluso sin préstamos) SELECT
u.nombre_usuario, COUNT(p.id_prestamo) AS prestamos
FROM usuarios u
LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario
GROUP BY u.id_usuario, u.nombre_usuario;
```

3. **Haces agregaciones sobre datos relacionados**

```
-- JOIN + GROUP BY: promedio de puntuaciones por autor
SELECT a.nombre_autor, ROUND(AVG(v.puntuacion), 2) AS puntuacion_media FROM
autores a
INNER JOIN libros l ON a.id_autor = l.id_autor
INNER JOIN valoraciones v ON l.id_libro = v.id_libro
GROUP BY a.id_autor, a.nombre_autor;
```

4. **La relación es 1:1 o 1:N sin duplicados problemáticos**

```
-- JOIN: cada préstamo tiene exactamente 1 usuario y 1 libro
SELECT u.nombre_usuario, l.titulo, p.fecha_prestamo
FROM prestamos p
INNER JOIN usuarios u ON p.id_usuario = u.id_usuario
INNER JOIN libros l ON p.id_libro = l.id_libro;
```

1.5.3 7.3. ¿Cuándo usar subconsultas?

✅ **Usa SUBCONSULTAS cuando:**

1. **Solo necesitas filtrar, no combinar columnas**

```
-- Subconsulta: solo quieres usuarios (no necesitas datos de préstamos) SELECT
nombre_usuario, email
FROM usuarios
WHERE id_usuario IN (SELECT id_usuario FROM prestamos);
```

2. **Necesitas cálculos agregados para filtrar**

```
-- Subconsulta: libros con más préstamos que el promedio
SELECT titulo, COUNT(*) AS veces_prestado
FROM libros l
INNER JOIN prestamos p ON l.id_libro = p.id_libro
GROUP BY l.id_libro, l.titulo
```

```
HAVING COUNT(*) > (
    SELECT AVG(prestamos_por_libro)
    FROM (
        SELECT COUNT(*) AS prestamos_por_libro
        FROM prestamos
        GROUP BY id_libro
    ) AS sub
);
```

3. **Usas NOT EXISTS para negaciones complejas**

```
-- Subconsulta: usuarios que NO han valorado ningún libro
SELECT nombre_usuario
```

(continúe en la próxima página)

(proviene de la página anterior)

```
FROM usuarios u
WHERE NOT EXISTS (
    SELECT 1
    FROM valoraciones v
    WHERE v.id_usuario = u.id_usuario
);
```

4. **Necesitas resultados en una sola columna para IN / NOT IN**

```
-- Subconsulta: libros de autores latinoamericanos
SELECT titulo
FROM libros
WHERE id_autor IN (
    SELECT id_autor
    FROM autores
    WHERE nacionalidad IN ('Colombiano', 'Argentino', 'Mexicano', 'Chileno') );
```

1.5.4 7.4. Casos donde son equivalentes

Ejemplo 42: Usuarios con multas pendientes

Opción A - JOIN:

```
SELECT DISTINCT u.nombre_usuario, u.email
FROM usuarios u
INNER JOIN prestamos p ON u.id_usuario = p.id_usuario
INNER JOIN multas m ON p.id_prestamo = m.id_prestamo
WHERE m.pagada = FALSE;
```

Opción B - Subconsulta con EXISTS:

```
SELECT u.nombre_usuario, u.email
FROM usuarios u
WHERE EXISTS (
    SELECT 1
    FROM prestamos p
    INNER JOIN multas m ON p.id_prestamo = m.id_prestamo
    WHERE p.id_usuario = u.id_usuario
      AND m.pagada = FALSE
);
```

Opción C - Subconsulta con IN:

```
SELECT u.nombre_usuario, u.email
FROM usuarios u
WHERE u.id_usuario IN (
    SELECT p.id_usuario
    FROM prestamos p
    INNER JOIN multas m ON p.id_prestamo = m.id_prestamo
    WHERE m.pagada = FALSE
);
```

Resultado esperado (idéntico en las 3):

nombre_usuario	email
Ana García	ana.garcia@email.com
Pedro Sánchez	pedro.sanchez@email.com

¿Cuál usar?

- **Opción A (JOIN):** Si necesitas datos de multas o préstamos
- **Opción B (EXISTS):** Más eficiente para verificar existencia (se detiene al encontrar el primero) • **Opción C (IN):** Más legible para filtros simples

1.5.5 7.5. Tabla comparativa con recomendaciones

Escenario	Mejor opción	Ejemplo
Necesitas columnas de 2+ ta blas	**JOIN**	Usuarios con nombre de libro prestado
Solo filtrar por existencia	**EXISTS**	Usuarios que hicieron préstamos
Filtrar por lista de IDs	**IN**	Libros de autores españoles
Agregaciones cruzadas	**JOIN + GROUP BY**	Promedio de valoraciones por autor
Comparar con prome dio/máximo	**Subconsulta en HAVING**	Libros más prestados que la media
Negaciones («que NO tie nen»)	**NOT EXISTS** o **LEFT JOIN + IS NULL**	Usuarios sin valoraciones préstamos)
Todos los registros (con/sin relación)	**LEFT JOIN**	Todos los usuarios (con o sin
Múltiples condiciones com plejas	**JOIN con WHERE**	Préstamos activos de libros publicados des pués de 2000

Ejemplo 43: NOT EXISTS vs LEFT JOIN + IS NULL

Objetivo: Libros que NO han sido valorados.

Opción A - NOT EXISTS:

```
SELECT l.titulo, a.nombre_autor
FROM libros l
INNER JOIN autores a ON l.id_autor = a.id_autor
WHERE NOT EXISTS (
    SELECT 1
    FROM valoraciones v
    WHERE v.id_libro = l.id_libro
```

);

Opción B - LEFT JOIN + IS NULL:

```
SELECT l.titulo, a.nombre_autor
FROM libros l
INNER JOIN autores a ON l.id_autor = a.id_autor
LEFT JOIN valoraciones v ON l.id_libro = v.id_libro
WHERE v.id_valoracion IS NULL;
```


Resultado esperado (idéntico):

titulo	nombre_autor
La metamorfosis	Franz Kafka
El amor en los tiempos del cólera	Gabriel García Márquez
Rayuela	Julio Cortázar
Rebelión en la granja	George Orwell

Rendimiento:

- NOT EXISTS → Se detiene al encontrar la primera coincidencia (más rápido)
- LEFT JOIN + IS NULL → Debe procesar todas las coincidencias

Recomendación: Usa NOT EXISTS para negaciones (más eficiente).

1.6 SECCIÓN 8: ERRORES COMUNES Y SOLUCIONES

1.6.1 8.1. Producto cartesiano accidental

2/7 ERROR: Olvidar la condición de JOIN o usar múltiples tablas sin unirlas.

Ejemplo 44: Producto cartesiano accidental

```
-- MAL: olvida conectar usuarios con prestamos
SELECT u.nombre_usuario, p.fecha_prestamo
FROM usuarios u, prestamos p; -- ¡7 usuarios × 20 préstamos = 140 filas sin sentido!
```

Resultado (primeras filas):

nombre_usuario	fecha_prestamo
Ana García	2024-11-15
Ana García	2024-11-18
Ana García	2024-11-20
Ana García	2024-11-22
Carlos López	2024-11-15
…	…

2/7 SOLUCIÓN:

```
-- BIEN: usa JOIN con condición
SELECT u.nombre_usuario, p.fecha_prestamo
FROM usuarios u
```

```
INNER JOIN prestamos p ON u.id_usuario = p.id_usuario;
```

Cómo detectarlo:

- El número de filas es sospechosamente alto (multiplicación de filas)
- Ves datos que «no deberían estar juntos»

1.6.2 8.2. Confundir ON con WHERE

2/7 ERROR: Poner condiciones de filtrado en ON (especialmente con LEFT JOIN).

Ejemplo 45: LEFT JOIN con filtro mal ubicado

```
-- MAL: filtro en ON excluye usuarios sin préstamos activos
SELECT u.nombre_usuario, COUNT(p.id_prestamo) AS activos
FROM usuarios u
LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario AND p.devuelto = FALSE GROUP BY
u.id_usuario, u.nombre_usuario;
```

Resultado:

nombre_usuario	activos
Ana García	1
Carlos López	0
David Ruiz	0
Elena Moreno	0
María Fernández	0
Pedro Sánchez	1

Esto ESTÁ BIEN si quieres contar activos de TODOS los usuarios.

Pero si haces:

```
-- MAL: filtro en WHERE convierte LEFT JOIN en INNER JOIN
SELECT u.nombre_usuario, p.fecha_prestamo
FROM usuarios u
LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario
WHERE p.devuelto = FALSE; -- ← elimina usuarios sin préstamos
```

Resultado:

nombre_usuario	fecha_prestamo
Ana García	2024-11-20
Pedro Sánchez	2024-11-25

2/7 Problema: David, Elena, Carlos y María desaparecen (no tienen préstamos activos).

2/7 SOLUCIÓN - Depende de lo que quieras:

```
-- Si quieres SOLO usuarios con préstamos activos → usa INNER JOIN SELECT
u.nombre_usuario, p.fecha_prestamo
FROM usuarios u
INNER JOIN prestamos p ON u.id_usuario = p.id_usuario
WHERE p.devuelto = FALSE;
```

```
-- Si quieres TODOS los usuarios, filtra en ON
SELECT u.nombre_usuario, p.fecha_prestamo
FROM usuarios u
LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario AND p.devuelto = FALSE;
```


DIDÁCTICO: JOINS EN MySQL

Base de datos a Datos: Especial Join

Regla:

- ON → Condiciones de **unión** (+ filtros sobre tabla derecha en LEFT JOIN)
- WHERE → Condiciones de **filtrado final** (aplican después del JOIN)

1.6.3 8.3. Manejo de NULL en JOINs externos

2/7 ERROR: No considerar valores NULL en LEFT/RIGHT JOIN.

Ejemplo 46: COUNT incorrecto con LEFT JOIN

```
-- MAL: COUNT(*) cuenta filas con NULL
SELECT u.nombre_usuario,
       COUNT(*) AS total -- ← cuenta 1 para usuarios sin préstamos
FROM usuarios u
LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario
GROUP BY u.id_usuario, u.nombre_usuario;
```

Resultado:

nombre_usuario	total
Ana García	5
Carlos López	2
David Ruiz	1
Elena Moreno	1
…	…

2/7 SOLUCIÓN:

```
-- BIEN: COUNT(columna) no cuenta NULL
SELECT u.nombre_usuario,
       COUNT(p.id_prestamo) AS prestamos_reales
FROM usuarios u
LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario
GROUP BY u.id_usuario, u.nombre_usuario;
```

Resultado correcto:

nombre_usuario	prestamos_reales
Ana García	5
Carlos López	2
David Ruiz	0
Elena Moreno	0
…	…

Alternativas para manejar NULL:

```
-- COALESCE: convierte NULL en 0
SELECT u.nombre_usuario,
       COALESCE(SUM(m.importe), 0) AS deuda_total
```

(continúe en la próxima página)

(proviene de la página anterior)

```
FROM usuarios u
LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario
LEFT JOIN multas m ON p.id_prestamo = m.id_prestamo
GROUP BY u.id_usuario, u.nombre_usuario;

-- IFNULL: similar a COALESCE (MySQL específico)
SELECT u.nombre_usuario,
       IFNULL(MAX(p.fecha_prestamo), 'Nunca') AS ultimo_prestamo
FROM usuarios u
LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario
GROUP BY u.id_usuario, u.nombre_usuario;
```

1.6.4 8.4. Columnas ambiguas sin cualificar

2/7 ERROR: No especificar de qué tabla viene una columna cuando aparece en ambas.

Ejemplo 47: Columna ambigua

```
-- ERROR: MySQL no sabe si te refieres a usuarios.id_usuario o prestamos.id_usuario SELECT nombre_usuario,
id_usuario
FROM usuarios u
INNER JOIN prestamos p ON u.id_usuario = p.id_usuario;
```

Error de MySQL:

```
ERROR 1052 (23000): Column 'id_usuario' in field list is ambiguous
```

2/7 SOLUCIÓN:

```
-- BIEN: cualifica con el alias de tabla
SELECT u.nombre_usuario, u.id_usuario
FROM usuarios u
INNER JOIN prestamos p ON u.id_usuario = p.id_usuario;
```

Buena práctica: Cualifica TODAS las columnas (incluso las no ambiguas) para claridad.

```
-- MEJOR: cualifica todas (más legible en consultas complejas)
SELECT u.nombre_usuario,
       u.email,
       p.fecha_prestamo,
       l.titulo
FROM usuarios u
INNER JOIN prestamos p ON u.id_usuario = p.id_usuario
INNER JOIN libros l ON p.id_libro = l.id_libro;
```

1.6.5 8.5. Problemas de rendimiento con JOINs

2/7 ERROR: JOINs muy lentos por falta de índices o diseño ineficiente.

Problema 1: Sin índices en claves foráneas

```
-- LENTO: si id_usuario en PRESTAMOS no tiene índice
SELECT u.nombre_usuario, COUNT(*) AS prestamos
FROM usuarios u
LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario
GROUP BY u.id_usuario, u.nombre_usuario;
```

2/7 SOLUCIÓN:

```
-- Crear índice en clave foránea
CREATE INDEX idx_prestamos_usuario ON prestamos(id_usuario);
CREATE INDEX idx_prestamos_libro ON prestamos(id_libro);
```

Regla: Toda clave foránea debe tener índice.

Problema 2: Múltiples JOINs sin filtrar antes

```
-- LENTO: une TODAS las tablas antes de filtrar
SELECT u.nombre_usuario, l.titulo
FROM usuarios u
INNER JOIN prestamos p ON u.id_usuario = p.id_usuario
INNER JOIN libros l ON p.id_libro = l.id_libro
INNER JOIN autores a ON l.id_autor = a.id_autor
WHERE a.nacionalidad = 'Español'
  AND p.fecha_prestamo > '2024-11-01';
```

2/7 SOLUCIÓN: Filtra temprano con subconsultas

```
-- RÁPIDO: filtra autores antes del JOIN
SELECT u.nombre_usuario, l.titulo
FROM usuarios u
INNER JOIN prestamos p ON u.id_usuario = p.id_usuario
INNER JOIN libros l ON p.id_libro = l.id_libro
INNER JOIN (
    SELECT id_autor
    FROM autores
    WHERE nacionalidad = 'Español'
) AS autores_esp ON l.id_autor = autores_esp.id_autor
WHERE p.fecha_prestamo > '2024-11-01';
```

Problema 3: SELECT * con múltiples JOINs

```
-- LENTO + gasta memoria: trae TODAS las columnas de 4 tablas
SELECT *
FROM usuarios u
INNER JOIN prestamos p ON u.id_usuario = p.id_usuario
INNER JOIN libros l ON p.id_libro = l.id_libro
INNER JOIN autores a ON l.id_autor = a.id_autor;
```

2/7 SOLUCIÓN:

```
-- RÁPIDO: solo columnas necesarias
SELECT u.nombre_usuario,
       l.titulo,
       a.nombre_autor,
       p.fecha_prestamo
FROM usuarios u
INNER JOIN prestamos p ON u.id_usuario = p.id_usuario
INNER JOIN libros l ON p.id_libro = l.id_libro
INNER JOIN autores a ON l.id_autor = a.id_autor;
```

Regla: Nunca uses SELECT * en producción (excepto para desarrollo rápido).

Verificar rendimiento con EXPLAIN

```
-- Ver cómo MySQL ejecuta la consulta
EXPLAIN SELECT u.nombre_usuario, COUNT(*) AS prestamos
FROM usuarios u
LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario
GROUP BY u.id_usuario;
```

Busca:

- type = ALL → Escaneo completo (malo, añade índice)
- type = index → Usa índice (bueno)
- type = ref → Usa índice con referencia (muy bueno)
- Extra = Using filesort → Ordenamiento sin índice (considera añadir índice en ORDER BY)

1.7 SECCIÓN 9: BUENAS PRÁCTICAS Y TABLA DE DECISIÓN

1.7.1 9.1. Checklist de buenas prácticas con JOINs

ANTES DE ESCRIBIR LA CONSULTA:

- [ ] **Identifica qué tablas necesitas** → Dibuja un mini-esquema en papel
- [ ] **Determina el tipo de JOIN** → ¿Necesitas todos los registros o solo coincidencias?
- [ ] **Verifica las claves foráneas** → ¿Qué columnas conectan las tablas?
- [ ] **Planifica el orden de las tablas** → Empieza por la tabla «central» de tu consulta

AL ESCRIBIR LA CONSULTA:

- [ ] **Usa alias cortos y claros** → usuarios u, libros l (no tabla1, x, t)
- [ ] **Cualifica TODAS las columnas** → u.nombre, no solo nombre (evita ambigüedad)
- [ ] **Usa JOIN explícito** → Nunca FROM tabla1, tabla2 WHERE ... (sintaxis antigua) • [

] **Separa ON de WHERE** → ON para unión, WHERE para filtrado

- [ ] **Indenta correctamente** → Cada JOIN en su línea, condiciones alineadas
- [ ] **Especifica columnas** → SELECT u.nombre, l.titulo, nunca SELECT *
- [ ] **LEFT JOIN con cuidado** → Recuerda que WHERE puede convertirlo en

INNER JOIN **Ejemplo de código bien formateado:**

```
SELECT u.nombre_usuario,
       l.titulo,
       a.nombre_autor,
       p.fecha_prestamo
FROM prestamos p
    INNER JOIN usuarios u ON p.id_usuario = u.id_usuario
    INNER JOIN libros l ON p.id_libro = l.id_libro
    INNER JOIN autores a ON l.id_autor = a.id_autor
WHERE p.fecha_prestamo >= '2024-11-01'
ORDER BY p.fecha_prestamo DESC;
```

DESPUÉS DE ESCRIBIR (REVISIÓN):

- [ ] **Prueba con datos reales** → Verifica que los resultados tengan sentido
- [ ] **Cuenta las filas** → ¿El número es razonable o hay producto cartesiano?
- [ ] **Verifica NULL** → En LEFT JOIN, ¿manejas correctamente los NULL?
- [ ] **Revisa duplicados** → ¿Necesitas DISTINCT o hay relaciones N:M sin controlar?
- [ ] **Comprueba rendimiento** → Usa EXPLAIN si la consulta es lenta
- [ ] **Documenta consultas complejas** → Añade comentarios en JOINs con 4+ tablas

OPTIMIZACIÓN Y RENDIMIENTO:

- [ ] **Índices en FKs** → Toda clave foránea debe tener índice
- [ ] **Filtra temprano** → Condiciones en WHERE lo antes posible

• [ ] **Evita funciones en ON** → ON YEAR(fecha) = 2024 es lento (mejor fecha >= '2024-01-01') • [ ] **Limita resultados** → Usa LIMIT en consultas exploratorias

• [ ] **Evita subconsultas correlacionadas** → Suelen ser más lentas que JOINs

1.7.2 9.2. Tabla de decisión: ¿Qué tipo de JOIN usar? Pregunta

clave Tipo de JOIN Ejemplo

¿Quiero **solo** registros con coincidencias en ambas tablas? **INNER JOIN** Préstamos activos (usuario + li bro)
¿Quiero **todos** los registros de la tabla A (con o sin B)? **LEFT JOIN** Todos los usuarios (incluso sin préstamos)
¿Quiero **todos** los registros de la tabla B (con o sin A)? **RIGHT JOIN** o **LEFT JOIN in vertido** Todos los libros (incluso sin prés tamos)
¿Quiero **todos** los registros de ambas ta blas? **FULL OUTER JOIN** (simulado con UNION) Usuarios sin préstamos + libros sin préstamos
¿Quiero **todas las combinaciones posi bles** de A × B? **CROSS JOIN** Salas × Turnos (planning comple to)
¿Quiero comparar filas **dentro de la mis ma tabla**? **SELF JOIN** Libros del mismo autor
¿Necesito encontrar registros **sin coinci dencias**? **LEFT JOIN + WHERE IS NULL** Usuarios que NO han hecho prés tamos

Diagrama de decisión visual:

```
┌─────────────────────────────────────────────────────┐
│ ¿Necesitas TODOS los registros de alguna │
│ tabla (incluso sin coincidencias)? │
└─────────────────────────────────────────────────────┘
                │
        NO │ SÍ
        ┌──────────────┐
        │ │
        ▼ ▼
INNER JOIN ¿De qué tabla?
(coincidencias) │
                ┌──────────┐
                │ │
            Tabla A Tabla B
                │ │
                ▼ ▼
            LEFT JOIN RIGHT JOIN
            (todas A) (todas B)
```

1.7.3 9.3. Recomendaciones de estilo y legibilidad

fi BUENAS PRÁCTICAS DE ESTILO:

1. Usa palabras clave en MAYÚSCULAS

```
-- BIEN: legible
SELECT u.nombre_usuario, l.titulo
FROM usuarios u
```

(continúe en la próxima página)

(proviene de la página anterior)

```
INNER JOIN libros l ON u.id_usuario = l.id_usuario;

-- MAL: difícil de leer
select u.nombre_usuario, l.titulo
from usuarios u
inner join libros l on u.id_usuario = l.id_usuario;
```

2. Una tabla/JOIN por línea

```
-- BIEN: estructura clara
SELECT u.nombre_usuario, l.titulo, a.nombre_autor
FROM prestamos p
    INNER JOIN usuarios u ON p.id_usuario = u.id_usuario
    INNER JOIN libros l ON p.id_libro = l.id_libro
    INNER JOIN autores a ON l.id_autor = a.id_autor;

-- MAL: todo en una línea
SELECT u.nombre_usuario, l.titulo FROM prestamos p INNER JOIN usuarios u ON p.id_usuario =
  u.id_usuario INNER JOIN libros l ON p.id_libro = l.id_libro;
```

3. Indenta subconsultas

```
-- BIEN: subconsulta indentada
SELECT u.nombre_usuario, prestamos_count
FROM usuarios u
    INNER JOIN (
        SELECT id_usuario, COUNT(*) AS prestamos_count
        FROM prestamos
        GROUP BY id_usuario
    ) AS stats ON u.id_usuario = stats.id_usuario;
```

4. Usa comentarios en JOINs complejos

```
SELECT u.nombre_usuario, l.titulo, m.importe
FROM usuarios u
    -- Usuario realizó préstamos
    INNER JOIN prestamos p ON u.id_usuario = p.id_usuario
    -- Préstamo asociado a un libro
    INNER JOIN libros l ON p.id_libro = l.id_libro
    -- Préstamo puede tener multa (opcional)
        LEFT JOIN multas m ON p.id_prestamo = m.id_prestamo AND m.pagada = FALSE;
```

5. Agrupa condiciones relacionadas

```
-- BIEN: condiciones agrupadas lógicamente
WHERE p.fecha_prestamo >= '2024-11-01'
  AND p.devuelto = FALSE
  AND (l.anio_publicacion > 2000 OR a.nacionalidad = 'Español');
```

1.7.4 9.4. Orden de ejecución y optimización

Orden lógico de ejecución de una consulta con JOINs:

```
SELECT u.nombre_usuario, COUNT(*) AS prestamos -- 5. Selecciona columnas FROM usuarios u -- 1.
Tabla base LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario -- 2. Aplica JOIN WHERE
p.fecha_prestamo > '2024-01-01' -- 3. Filtra filas GROUP BY u.id_usuario, u.nombre_usuario -- 4.
Agrupa
HAVING COUNT(*) > 2 -- 6. Filtra grupos ORDER BY prestamos DESC -- 7. Ordena LIMIT 10; -- 8.
Limita resultados
```

Orden real de ejecución:

1. **FROM** → Identifica tablas
2. **JOIN** → Une tablas (construye conjunto de datos temporal)
3. **WHERE** → Filtra filas individuales
4. **GROUP BY** → Agrupa filas
5. **HAVING** → Filtra grupos
6. **SELECT** → Selecciona columnas (aquí se evalúan alias)
7. **ORDER BY** → Ordena resultados
8. **LIMIT** → Limita número de filas devueltas

Optimización basada en orden de ejecución:

2/7 INEFICIENTE: Filtrar después de agregar

```
-- Agrupa TODOS los usuarios, luego filtra
SELECT u.nombre_usuario, COUNT(*) AS prestamos
FROM usuarios u
    LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario
GROUP BY u.id_usuario, u.nombre_usuario
HAVING COUNT(*) > 5; -- Filtro tardío
```

2/7 EFICIENTE: Filtrar antes de agregar

```
-- Filtra primero, luego agrupa menos datos
SELECT u.nombre_usuario, COUNT(*) AS prestamos
FROM usuarios u
    LEFT JOIN (
        SELECT id_usuario, id_prestamo
        FROM prestamos
        WHERE fecha_prestamo >= '2024-01-01' -- Filtro temprano
    ) p ON u.id_usuario = p.id_usuario
GROUP BY u.id_usuario, u.nombre_usuario
HAVING COUNT(*) > 5;
```

Trucos de optimización avanzados:

1. Usa índices compuestos para JOINs + WHERE

```
-- Si haces esto frecuentemente:
SELECT u.nombre_usuario, l.titulo
FROM prestamos p
    INNER JOIN usuarios u ON p.id_usuario = u.id_usuario
    INNER JOIN libros l ON p.id_libro = l.id_libro
WHERE p.devuelto = FALSE;

-- Crea este índice:
CREATE INDEX idx_prestamos_devuelto_usuario_libro
ON prestamos(devuelto, id_usuario, id_libro);
```

2. Considera vistas materializadas para consultas repetitivas

```
-- Si calculas esto muchas veces, crea una vista
CREATE VIEW vista_prestamos_completos AS
SELECT u.nombre_usuario,
        l.titulo,
        a.nombre_autor,
        p.fecha_prestamo,
        p.devuelto
FROM prestamos p
    INNER JOIN usuarios u ON p.id_usuario = u.id_usuario
    INNER JOIN libros l ON p.id_libro = l.id_libro
    INNER JOIN autores a ON l.id_autor = a.id_autor;

-- Luego úsala directamente:
SELECT * FROM vista_prestamos_completos WHERE devuelto = FALSE;
```

1.8 SECCIÓN 10: EJERCICIOS PRÁCTICOS CLASIFICADOS

Instrucciones generales:

- Usa la base de datos de biblioteca creada en este manual
- Escribe las consultas sin mirar las soluciones
- Verifica tus resultados ejecutándolas en MySQL
- Compara tu enfoque con la solución propuesta

1.8.1 10.1. Nivel Básico - INNER JOIN simple (10 ejercicios)

Ejercicio 1: Listar todos los préstamos mostrando nombre del usuario y título del libro.

Tablas: PRESTAMOS, USUARIOS, LIBROS
JOIN necesario: INNER JOIN (2 veces)

Dificultad: ⭐☆☆☆☆

PRESTAMOS tiene id_usuario (FK → USUARIOS) y id_libro (FK → LIBROS)

```
SELECT u.nombre_usuario,
       l.titulo,
       p.fecha_prestamo
FROM prestamos p
    INNER JOIN usuarios u ON p.id_usuario = u.id_usuario
    INNER JOIN libros l ON p.id_libro = l.id_libro
ORDER BY p.fecha_prestamo DESC;
```

Ejercicio 2: Mostrar todos los libros con el nombre de su autor.

Tablas: LIBROS, AUTORES
JOIN necesario: INNER JOIN
Dificultad: ⭐☆☆☆☆

```
SELECT l.titulo,
       a.nombre_autor,
       l.anio_publicacion
FROM libros l
    INNER JOIN autores a ON l.id_autor = a.id_autor
ORDER BY a.nombre_autor, l.titulo;
```

Ejercicio 3: Listar las valoraciones mostrando quién valoró qué libro.

Tablas: VALORACIONES, USUARIOS, LIBROS
JOIN necesario: INNER JOIN (2 veces)
Dificultad: ⭐☆☆☆☆

```
SELECT u.nombre_usuario,
       l.titulo,
       v.puntuacion,
       v.comentario,
       v.fecha_valoracion
FROM valoraciones v
    INNER JOIN usuarios u ON v.id_usuario = u.id_usuario
    INNER JOIN libros l ON v.id_libro = l.id_libro
ORDER BY v.fecha_valoracion DESC;
```

Ejercicio 4: Mostrar préstamos con multa, indicando usuario, libro e importe de la multa.

Tablas: PRESTAMOS, USUARIOS, LIBROS, MULTAS
JOIN necesario: INNER JOIN (3 veces)
Dificultad: ⭐⭐☆☆☆

```
SELECT u.nombre_usuario,
       l.titulo,
       p.fecha_prestamo,
       m.importe AS multa
FROM prestamos p
    INNER JOIN usuarios u ON p.id_usuario = u.id_usuario
    INNER JOIN libros l ON p.id_libro = l.id_libro
```

(continúe en la próxima página)

(proviene de la página anterior)

```
    INNER JOIN multas m ON p.id_prestamo = m.id_prestamo
WHERE m.pagada = FALSE
ORDER BY m.importe DESC;
```

Ejercicio 5: Listar libros con su autor y editorial.

Tablas: LIBROS, AUTORES, EDITORIALES
JOIN necesario: INNER JOIN (2 veces)
Dificultad: ⭐☆☆☆☆

```
SELECT l.titulo,
       a.nombre_autor,
       e.nombre_editorial,
       l.anio_publicacion
FROM libros l
    INNER JOIN autores a ON l.id_autor = a.id_autor
    INNER JOIN editoriales e ON l.id_editorial = e.id_editorial
ORDER BY e.nombre_editorial, a.nombre_autor;
```

Ejercicio 6: Mostrar préstamos activos (no devueltos) con nombre de usuario.

Tablas: PRESTAMOS, USUARIOS
JOIN necesario: INNER JOIN
Dificultad: ⭐☆☆☆☆

```
SELECT u.nombre_usuario,
       u.email,
       p.fecha_prestamo,
       DATEDIFF(CURDATE(), p.fecha_prestamo) AS dias_prestado
FROM prestamos p
    INNER JOIN usuarios u ON p.id_usuario = u.id_usuario
WHERE p.devuelto = FALSE
ORDER BY dias_prestado DESC;
```

Ejercicio 7: Listar autores españoles con sus libros.

Tablas: AUTORES, LIBROS
JOIN necesario: INNER JOIN
Dificultad: ⭐☆☆☆☆

```
SELECT a.nombre_autor,
       l.titulo,
       l.anio_publicacion
FROM autores a
    INNER JOIN libros l ON a.id_autor = l.id_autor
WHERE a.nacionalidad = 'Español'
ORDER BY a.nombre_autor, l.anio_publicacion;
```

Ejercicio 8: Mostrar valoraciones de 5 estrellas con título del libro y nombre del usuario.

Tablas: VALORACIONES, LIBROS, USUARIOS
JOIN necesario: INNER JOIN (2 veces)
Dificultad: ⭐☆☆☆☆

```
SELECT l.titulo,
       u.nombre_usuario,
       v.comentario,
       v.fecha_valoracion
FROM valoraciones v
    INNER JOIN libros l ON v.id_libro = l.id_libro
    INNER JOIN usuarios u ON v.id_usuario = u.id_usuario
WHERE v.puntuacion = 5
ORDER BY v.fecha_valoracion DESC;
```

Ejercicio 9: Listar libros publicados después de 2000 con su autor.

Tablas: LIBROS, AUTORES
JOIN necesario: INNER JOIN
Dificultad: ⭐☆☆☆☆

```
SELECT l.titulo,
       a.nombre_autor,
       l.anio_publicacion
FROM libros l
    INNER JOIN autores a ON l.id_autor = a.id_autor
WHERE l.anio_publicacion > 2000
ORDER BY l.anio_publicacion DESC;
```

Ejercicio 10: Mostrar préstamos de noviembre de 2024 con usuario y libro.

Tablas: PRESTAMOS, USUARIOS, LIBROS
JOIN necesario: INNER JOIN (2 veces)
Dificultad: ⭐⭐☆☆☆

```
SELECT u.nombre_usuario,
       l.titulo,
       p.fecha_prestamo
FROM prestamos p
    INNER JOIN usuarios u ON p.id_usuario = u.id_usuario
    INNER JOIN libros l ON p.id_libro = l.id_libro
WHERE p.fecha_prestamo >= '2024-11-01'
  AND p.fecha_prestamo < '2024-12-01'
ORDER BY p.fecha_prestamo;
```

1.8.2 10.2. Nivel Intermedio - LEFT/RIGHT JOIN (10 ejercicios)

Ejercicio 11: Listar TODOS los usuarios con el número de préstamos que han hecho (incluso si es 0).

Tablas: USUARIOS, PRESTAMOS
JOIN necesario: LEFT JOIN + COUNT
Dificultad: ⭐⭐☆☆☆

Usa LEFT JOIN y COUNT(p.id_prestamo) para no contar NULL

```
SELECT u.nombre_usuario,
       u.email,
       COUNT(p.id_prestamo) AS total_prestamos
FROM usuarios u
    LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario
GROUP BY u.id_usuario, u.nombre_usuario, u.email
ORDER BY total_prestamos DESC;
```

Ejercicio 12: Encontrar usuarios que NUNCA han hecho un préstamo.

Tablas: USUARIOS, PRESTAMOS
JOIN necesario: LEFT JOIN + WHERE IS NULL
Dificultad: ⭐⭐☆☆☆

```
SELECT u.nombre_usuario,
       u.email,
       u.fecha_registro
FROM usuarios u
    LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario
WHERE p.id_prestamo IS NULL;
```

Ejercicio 13: Mostrar TODOS los libros con el número de veces que han sido prestados.

Tablas: LIBROS, PRESTAMOS, AUTORES
JOIN necesario: INNER JOIN (autor) + LEFT JOIN (préstamos)
Dificultad: ⭐⭐⭐☆☆

```
SELECT l.titulo,
       a.nombre_autor,
       COUNT(p.id_prestamo) AS veces_prestado
FROM libros l
    INNER JOIN autores a ON l.id_autor = a.id_autor
    LEFT JOIN prestamos p ON l.id_libro = p.id_libro
GROUP BY l.id_libro, l.titulo, a.nombre_autor
ORDER BY veces_prestado DESC, l.titulo;
```

Ejercicio 14: Encontrar libros que NUNCA han sido prestados.

Tablas: LIBROS, PRESTAMOS, AUTORES
JOIN necesario: INNER JOIN + LEFT JOIN + WHERE IS NULL
Dificultad: ⭐⭐☆☆☆

```
SELECT l.titulo,
       a.nombre_autor,
       l.anio_publicacion
FROM libros l
    INNER JOIN autores a ON l.id_autor = a.id_autor
    LEFT JOIN prestamos p ON l.id_libro = p.id_libro
WHERE p.id_prestamo IS NULL
ORDER BY a.nombre_autor, l.titulo;
```

Ejercicio 15: Listar TODOS los autores con el número de libros que tienen publicados.

Tablas: AUTORES, LIBROS
JOIN necesario: LEFT JOIN + COUNT
Dificultad: ⭐⭐☆☆☆

```
SELECT a.nombre_autor,
       a.nacionalidad,
       COUNT(l.id_libro) AS libros_publicados
FROM autores a
    LEFT JOIN libros l ON a.id_autor = l.id_autor
GROUP BY a.id_autor, a.nombre_autor, a.nacionalidad
ORDER BY libros_publicados DESC, a.nombre_autor;
```

Ejercicio 16: Mostrar TODOS los usuarios con su deuda total en multas (0 si no tienen).

Tablas: USUARIOS, PRESTAMOS, MULTAS
JOIN necesario: LEFT JOIN (2 veces) + COALESCE
Dificultad: ⭐⭐⭐☆☆

```
SELECT u.nombre_usuario,
       COALESCE(SUM(m.importe), 0) AS deuda_total,
       COUNT(m.id_multa) AS num_multas_pendientes
FROM usuarios u
    LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario
    LEFT JOIN multas m ON p.id_prestamo = m.id_prestamo AND m.pagada = FALSE GROUP BY
u.id_usuario, u.nombre_usuario
ORDER BY deuda_total DESC;
```

Ejercicio 17: Encontrar autores que NO tienen libros publicados en la biblioteca.

Tablas: AUTORES, LIBROS
JOIN necesario: LEFT JOIN + WHERE IS NULL
Dificultad: ⭐⭐☆☆☆

```
SELECT a.nombre_autor,
       a.nacionalidad
FROM autores a
    LEFT JOIN libros l ON a.id_autor = l.id_autor
WHERE l.id_libro IS NULL;
```

Ejercicio 18: Listar TODOS los libros con su puntuación promedio (NULL si no tienen valoraciones).

Tablas: LIBROS, VALORACIONES, AUTORES
JOIN necesario: INNER JOIN + LEFT JOIN + AVG
Dificultad: ⭐⭐⭐☆☆

```
SELECT l.titulo,
       a.nombre_autor,
       COUNT(v.id_valoracion) AS num_valoraciones,
       ROUND(AVG(v.puntuacion), 2) AS puntuacion_media
FROM libros l
    INNER JOIN autores a ON l.id_autor = a.id_autor
    LEFT JOIN valoraciones v ON l.id_libro = v.id_libro
GROUP BY l.id_libro, l.titulo, a.nombre_autor
ORDER BY puntuacion_media DESC NULLS LAST, l.titulo;
```

Ejercicio 19: Mostrar usuarios con préstamos activos y el total de días que llevan prestados.

Tablas: USUARIOS, PRESTAMOS, LIBROS
JOIN necesario: LEFT JOIN con condición en ON
Dificultad: ⭐⭐⭐☆☆

```
SELECT u.nombre_usuario,
       l.titulo,
       p.fecha_prestamo,
       DATEDIFF(CURDATE(), p.fecha_prestamo) AS dias_prestado
FROM usuarios u
    LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario AND p.devuelto = FALSE LEFT JOIN libros
    l ON p.id_libro = l.id_libro
WHERE p.id_prestamo IS NOT NULL
ORDER BY dias_prestado DESC;
```

Ejercicio 20: Listar libros con el número de valoraciones y préstamos (ambos pueden ser 0).

Tablas: LIBROS, VALORACIONES, PRESTAMOS, AUTORES
JOIN necesario: INNER JOIN + LEFT JOIN (2 veces)
Dificultad: ⭐⭐⭐☆☆

```
SELECT l.titulo,
       a.nombre_autor,
       COUNT(DISTINCT v.id_valoracion) AS num_valoraciones,
       COUNT(DISTINCT p.id_prestamo) AS num_prestamos
FROM libros l
    INNER JOIN autores a ON l.id_autor = a.id_autor
    LEFT JOIN valoraciones v ON l.id_libro = v.id_libro
    LEFT JOIN prestamos p ON l.id_libro = p.id_libro
GROUP BY l.id_libro, l.titulo, a.nombre_autor
ORDER BY num_prestamos DESC, num_valoraciones DESC;
```

1.8.3 10.3. Nivel Avanzado - Múltiples JOINs y agregaciones (8 ejercicios)

Ejercicio 21: Top 5 usuarios con más préstamos, mostrando cuántos están activos y cuántos de vueltos.

Tablas: USUARIOS, PRESTAMOS
JOIN necesario: INNER JOIN + COUNT con CASE
Dificultad: ⭐⭐⭐☆☆

```
SELECT u.nombre_usuario,
       COUNT(*) AS total_prestamos,
       SUM(CASE WHEN p.devuelto = FALSE THEN 1 ELSE 0 END) AS activos,
       SUM(CASE WHEN p.devuelto = TRUE THEN 1 ELSE 0 END) AS devueltos
FROM usuarios u
    INNER JOIN prestamos p ON u.id_usuario = p.id_usuario
GROUP BY u.id_usuario, u.nombre_usuario
ORDER BY total_prestamos DESC
LIMIT 5;
```

Ejercicio 22: Autores con mejor puntuación promedio (mínimo 3 valoraciones).

Tablas: AUTORES, LIBROS, VALORACIONES
JOIN necesario: INNER JOIN (2 veces) + HAVING
Dificultad: ⭐⭐⭐☆☆

```
SELECT a.nombre_autor,
       COUNT(DISTINCT l.id_libro) AS libros_en_bd,
       COUNT(v.id_valoracion) AS total_valoraciones,
```

```
        ROUND(AVG(v.puntuacion), 2) AS puntuacion_media
FROM autores a
    INNER JOIN libros l ON a.id_autor = l.id_autor
    INNER JOIN valoraciones v ON l.id_libro = v.id_libro
GROUP BY a.id_autor, a.nombre_autor
HAVING COUNT(v.id_valoracion) >= 3
ORDER BY puntuacion_media DESC;
```


Ejercicio 23: Libros más populares del último mes (por préstamos).

Tablas: LIBROS, PRESTAMOS, AUTORES
JOIN necesario: INNER JOIN (2 veces) + WHERE fecha
Dificultad: ⭐⭐⭐☆☆

```
SELECT l.titulo,
        a.nombre_autor,
        COUNT(*) AS prestamos_ultimo_mes
FROM prestamos p
    INNER JOIN libros l ON p.id_libro = l.id_libro
    INNER JOIN autores a ON l.id_autor = a.id_autor
WHERE p.fecha_prestamo >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH)
GROUP BY l.id_libro, l.titulo, a.nombre_autor
ORDER BY prestamos_ultimo_mes DESC
LIMIT 10;
```

Ejercicio 24: Usuarios morosos (con multas pendientes superior a 10€).

Tablas: USUARIOS, PRESTAMOS, MULTAS
JOIN necesario: INNER JOIN (2 veces) + HAVING
Dificultad: ⭐⭐⭐☆☆

```
SELECT u.nombre_usuario,
        u.email,
        SUM(m.importe) AS deuda_total,
        COUNT(m.id_multa) AS num_multas_pendientes
FROM usuarios u
    INNER JOIN prestamos p ON u.id_usuario = p.id_usuario
    INNER JOIN multas m ON p.id_prestamo = m.id_prestamo
WHERE m.pagada = FALSE
GROUP BY u.id_usuario, u.nombre_usuario, u.email
HAVING SUM(m.importe) > 10
ORDER BY deuda_total DESC;
```

Ejercicio 25: Libros con valoraciones contradictorias (tienen tanto 5 como 1-2 estrellas).

Tablas: LIBROS, VALORACIONES, AUTORES
JOIN necesario: INNER JOIN + HAVING con MIN/MAX
Dificultad: ⭐⭐⭐⭐☆

```
SELECT l.titulo,
        a.nombre_autor,
        MIN(v.puntuacion) AS peor_valoracion,
        MAX(v.puntuacion) AS mejor_valoracion,
        ROUND(AVG(v.puntuacion), 2) AS promedio
FROM libros l
    INNER JOIN autores a ON l.id_autor = a.id_autor
```

```
    INNER JOIN valoraciones v ON l.id_libro = v.id_libro
GROUP BY l.id_libro, l.titulo, a.nombre_autor
HAVING MAX(v.puntuacion) = 5 AND MIN(v.puntuacion) <= 2
ORDER BY promedio;
```


Ejercicio 26: Parejas de usuarios que prestaron los mismos libros (SELF JOIN).

Tablas: PRESTAMOS (consigo misma), USUARIOS (2 veces), LIBROS
JOIN necesario: SELF JOIN + múltiples INNER JOINs
Dificultad: ⭐⭐⭐⭐☆

```
SELECT DISTINCT u1.nombre_usuario AS usuario1,
        u2.nombre_usuario AS usuario2,
        l.titulo AS libro_en_comun
FROM prestamos p1
    INNER JOIN prestamos p2 ON p1.id_libro = p2.id_libro
                                AND p1.id_usuario < p2.id_usuario
    INNER JOIN usuarios u1 ON p1.id_usuario = u1.id_usuario
    INNER JOIN usuarios u2 ON p2.id_usuario = u2.id_usuario
    INNER JOIN libros l ON p1.id_libro = l.id_libro
ORDER BY l.titulo, u1.nombre_usuario;
```

Ejercicio 27: Informe mensual: préstamos, nuevas valoraciones y multas generadas por mes.

Tablas: PRESTAMOS, VALORACIONES, MULTAS
JOIN necesario: UNION de 3 agregaciones
Dificultad: ⭐⭐⭐⭐☆

```
SELECT DATE_FORMAT(fecha_prestamo, '%Y-%m') AS mes,
        COUNT(*) AS prestamos,
        0 AS valoraciones,
        0 AS multas
FROM prestamos
GROUP BY DATE_FORMAT(fecha_prestamo, '%Y-%m')

UNION ALL

SELECT DATE_FORMAT(fecha_valoracion, '%Y-%m') AS mes,
        0 AS prestamos,
        COUNT(*) AS valoraciones,
        0 AS multas
FROM valoraciones
GROUP BY DATE_FORMAT(fecha_valoracion, '%Y-%m')

UNION ALL

SELECT DATE_FORMAT(p.fecha_prestamo, '%Y-%m') AS mes,
        0 AS prestamos,
        0 AS valoraciones,
        COUNT(m.id_multa) AS multas
FROM multas m
    INNER JOIN prestamos p ON m.id_prestamo = p.id_prestamo
GROUP BY DATE_FORMAT(p.fecha_prestamo, '%Y-%m')

ORDER BY mes DESC;
```

Ejercicio 28: Libros «olvidados» de autores populares (autor con ≥3 libros, pero este libro nunca prestado).

Tablas: LIBROS, AUTORES, PRESTAMOS (subconsulta)
JOIN necesario: INNER JOIN + subconsulta + LEFT JOIN + IS NULL
Dificultad: ⭐⭐⭐⭐☆

```
SELECT l.titulo,
a.nombre_autor,
l.anio_publicacion
FROM libros l
INNER JOIN autores a ON l.id_autor = a.id_autor
INNER JOIN (
SELECT id_autor
FROM libros
GROUP BY id_autor
HAVING COUNT(*) >= 3
) AS autores_populares ON l.id_autor = autores_populares.id_autor
LEFT JOIN prestamos p ON l.id_libro = p.id_libro
WHERE p.id_prestamo IS NULL
ORDER BY a.nombre_autor, l.titulo;
```

1.8.4 10.4. Nivel Experto - Casos complejos reales (5 ejercicios)

Ejercicio 29: Dashboard completo de un usuario: préstamos totales, activos, multas, valoraciones dadas.

Tablas: Todas las relevantes
JOIN necesario: Múltiples LEFT JOINs + agregaciones
Dificultad: ⭐⭐⭐⭐⭐

```
SELECT u.nombre_usuario,
u.email,
u.fecha_registro,
COUNT(DISTINCT p.id_prestamo) AS total_prestamos,
COUNT(DISTINCT CASE WHEN p.devuelto = FALSE THEN p.id_prestamo END) AS␣ ↪prestamos_activos,
COUNT(DISTINCT v.id_valoracion) AS valoraciones_dadas,
ROUND(AVG(v.puntuacion), 2) AS puntuacion_promedio_dada,
COUNT(DISTINCT m.id_multa) AS multas_pendientes,
COALESCE(SUM(CASE WHEN m.pagada = FALSE THEN m.importe ELSE 0 END), 0) AS␣ ↪deuda_total
FROM usuarios u
LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario
LEFT JOIN valoraciones v ON u.id_usuario = v.id_usuario
LEFT JOIN multas m ON p.id_prestamo = m.id_prestamo
WHERE u.nombre_usuario = 'Ana García' -- Cambiar por parámetro
```

```
GROUP BY u.id_usuario, u.nombre_usuario, u.email, u.fecha_registro;
```
1.8. SECCIÓN 10: EJERCICIOS

PRÁCTICOS CLASIFICADOS 53

Base de datos a Datos: Especial Join

Ejercicio 30: Análisis de «afinidad» de usuarios: usuarios con gustos similares basados en valora ciones comunes.

Tablas: VALORACIONES (SELF JOIN), USUARIOS (2 veces), LIBROS
JOIN necesario: SELF JOIN complejo + múltiples condiciones
Dificultad: ⭐⭐⭐⭐⭐

```
SELECT u1.nombre_usuario AS usuario1,
       u2.nombre_usuario AS usuario2,
       COUNT(*) AS libros_valorados_en_comun,
       ROUND(AVG(ABS(v1.puntuacion - v2.puntuacion)), 2) AS diferencia_promedio FROM
valoraciones v1
    INNER JOIN valoraciones v2 ON v1.id_libro = v2.id_libro
                                  AND v1.id_usuario < v2.id_usuario
    INNER JOIN usuarios u1 ON v1.id_usuario = u1.id_usuario
    INNER JOIN usuarios u2 ON v2.id_usuario = u2.id_usuario
GROUP BY u1.id_usuario, u2.id_usuario, u1.nombre_usuario, u2.nombre_usuario HAVING
COUNT(*) >= 2
ORDER BY libros_valorados_en_comun DESC, diferencia_promedio ASC;
```

Ejercicio 31: Ranking de «eficiencia» de biblioteca: libros con mejor ratio valoración/préstamos.

Tablas: LIBROS, AUTORES, PRESTAMOS, VALORACIONES
JOIN necesario: Múltiples JOINs + cálculo de ratio
Dificultad: ⭐⭐⭐⭐☆

```
SELECT l.titulo,
       a.nombre_autor,
       COUNT(DISTINCT p.id_prestamo) AS veces_prestado,
       COUNT(DISTINCT v.id_valoracion) AS num_valoraciones,
       ROUND(AVG(v.puntuacion), 2) AS puntuacion_media,
       ROUND(COUNT(DISTINCT v.id_valoracion) / NULLIF(COUNT(DISTINCT p.id_prestamo), 0) * 100,
  2) AS porcentaje_valoracion
FROM libros l
    INNER JOIN autores a ON l.id_autor = a.id_autor
    LEFT JOIN prestamos p ON l.id_libro = p.id_libro
    LEFT JOIN valoraciones v ON l.id_libro = v.id_libro
GROUP BY l.id_libro, l.titulo, a.nombre_autor
```

```
HAVING veces_prestado > 0
ORDER BY porcentaje_valoracion DESC, puntuacion_media DESC
LIMIT 10;
```

Ejercicio 32: Detectar préstamos sospechosos: mismo libro prestado a múltiples usuarios en fechas superpuestas.

Tablas: PRESTAMOS (SELF JOIN), LIBROS
JOIN necesario: SELF JOIN con condiciones de rango de fechas
Dificultad: ⭐⭐⭐⭐⭐

```
SELECT l.titulo,
        p1.id_prestamo AS prestamo1,
        p1.fecha_prestamo AS inicio1,
        p1.fecha_devolucion AS fin1,
        p2.id_prestamo AS prestamo2,
```

(continúe en la próxima página)

(proviene de la página anterior)

```
        p2.fecha_prestamo AS inicio2,
        p2.fecha_devolucion AS fin2,
        DATEDIFF(
            LEAST(COALESCE(p1.fecha_devolucion, CURDATE()),
                  COALESCE(p2.fecha_devolucion, CURDATE())),
            GREATEST(p1.fecha_prestamo, p2.fecha_prestamo)
        ) AS dias_solapados
FROM prestamos p1
    INNER JOIN prestamos p2 ON p1.id_libro = p2.id_libro
                                AND p1.id_prestamo < p2.id_prestamo
                                AND p1.fecha_prestamo <= COALESCE(p2.fecha_devolucion,
 CURDATE())
                                AND p2.fecha_prestamo <= COALESCE(p1.fecha_devolucion,
 CURDATE())
    INNER JOIN libros l ON p1.id_libro = l.id_libro
ORDER BY dias_solapados DESC, l.titulo;
```

Ejercicio 33: Reporte ejecutivo completo: estadísticas generales de la biblioteca.

Tablas: Todas
JOIN necesario: Subconsultas independientes combinadas
Dificultad: ⭐⭐⭐⭐⭐

```
SELECT
    (SELECT COUNT(*) FROM usuarios) AS total_usuarios,
    (SELECT COUNT(*) FROM usuarios u
     LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario
     WHERE p.id_prestamo IS NULL) AS usuarios_sin_prestamos,
    (SELECT COUNT(*) FROM libros) AS total_libros,
    (SELECT COUNT(DISTINCT id_libro) FROM prestamos) AS libros_alguna_vez_prestados, (SELECT
 COUNT(*) FROM prestamos WHERE devuelto = FALSE) AS prestamos_activos, (SELECT COUNT(*) FROM
 prestamos WHERE devuelto = TRUE) AS prestamos_devueltos, (SELECT ROUND(AVG(puntuacion), 2)
 FROM valoraciones) AS valoracion_promedio_ global,
    (SELECT SUM(importe) FROM multas WHERE pagada = FALSE) AS deuda_total_multas, (SELECT
    COUNT(*) FROM multas WHERE pagada = FALSE) AS multas_pendientes, (SELECT nombre_autor
    FROM autores a
     INNER JOIN libros l ON a.id_autor = l.id_autor
     INNER JOIN prestamos p ON l.id_libro = p.id_libro
     GROUP BY a.id_autor, a.nombre_autor
```

```
ORDER BY COUNT(*) DESC LIMIT 1) AS autor_mas_prestado;
```

1.9 SECCIÓN 11: CASOS PRÁCTICOS DE NEGOCIO

Objetivo: Aplicar JOINs a situaciones reales que te encontrarías trabajando como desarrollador en una biblioteca digital. Cada caso incluye:

- **Contexto empresarial**
- **Objetivo del informe**
- **Análisis del problema**
- **Solución SQL completa**

- **Interpretación de resultados**

1.9.1 CASO 1: Sistema de recomendaciones personalizado

Contexto:

El departamento de marketing quiere implementar un sistema de recomendaciones. Necesitan saber qué libros recomendar a cada usuario basándose en:

- Libros similares a los que ya han leído
- Libros populares entre usuarios con gustos similares
- Libros bien valorados del mismo autor

Objetivo:

Generar recomendaciones para el usuario «Ana García» (id_usuario = 1).

Análisis del problema:

Paso 1: Identificar autores que le gustan a Ana (basado en sus valoraciones altas) **Paso 2:** Encontrar libros de esos autores que NO ha leído **Paso 3:** Priorizar por popularidad (número de préstamos) y valoración promedio

Solución SQL:

```
-- PARTE 1: Autores favoritos de Ana (valoración >= 4)
WITH autores_favoritos_ana AS (
    SELECT DISTINCT l.id_autor
    FROM valoraciones v
        INNER JOIN libros l ON v.id_libro = l.id_libro
    WHERE v.id_usuario = 1 -- Ana García
      AND v.puntuacion >= 4
),

-- PARTE 2: Libros que Ana NO ha prestado ni valorado
libros_no_leidos AS (
    SELECT l.id_libro, l.titulo, l.id_autor
    FROM libros l
```

```
        WHERE l.id_libro NOT IN (
            SELECT id_libro FROM prestamos WHERE id_usuario = 1
        )
        AND l.id_libro NOT IN (
            SELECT id_libro FROM valoraciones WHERE id_usuario = 1
        )
),

-- PARTE 3: Estadísticas de cada libro candidato
estadisticas_libros AS (
        SELECT l.id_libro,
                COUNT(DISTINCT p.id_prestamo) AS veces_prestado,
                COUNT(DISTINCT v.id_valoracion) AS num_valoraciones,
                ROUND(AVG(v.puntuacion), 2) AS valoracion_media
```

(continúe en la próxima página)

(proviene de la página anterior)

```
        FROM libros l
            LEFT JOIN prestamos p ON l.id_libro = p.id_libro
            LEFT JOIN valoraciones v ON l.id_libro = v.id_libro
        GROUP BY l.id_libro
)

-- CONSULTA FINAL: Recomendaciones priorizadas
SELECT ln.titulo,
        a.nombre_autor,
        e.nombre_editorial,
        l.anio_publicacion,
        el.veces_prestado,
        el.num_valoraciones,
        COALESCE(el.valoracion_media, 0) AS valoracion_media,
        CASE
                        WHEN ln.id_autor IN (SELECT id_autor FROM autores_favoritos_ana)
            THEN 'Autor favorito '
            ELSE 'Nuevo autor'
        END AS categoria,
        -- Puntuación de recomendación (0-100)
        ROUND(
            (COALESCE(el.valoracion_media, 0) * 15) + -- Hasta 75 puntos por␣ ↪valoración
                (el.veces_prestado * 2) + -- 2 puntos por préstamo CASE WHEN ln.id_autor IN (SELECT
                id_autor FROM autores_favoritos_ana) THEN 10 ELSE 0 END -- 10 puntos bonus autor␣
↪favorito
        , 0) AS score_recomendacion
FROM libros_no_leidos ln
    INNER JOIN libros l ON ln.id_libro = l.id_libro
    INNER JOIN autores a ON ln.id_autor = a.id_autor
    INNER JOIN editoriales e ON l.id_editorial = e.id_editorial
    LEFT JOIN estadisticas_libros el ON ln.id_libro = el.id_libro
ORDER BY score_recomendacion DESC, valoracion_media DESC
LIMIT 5;
```

Resultado esperado:

titulo	nombre_autor	valoracion_media	veces_prestado	categoria	score_recomendacion
El otoño del patriarca	Gabriel García Márquez	4.50	2	Autor favorito ⭐	82
Rayuela	Julio Cortázar	4.00	3	Nuevo autor	66
…	…	…	…	…	…

Interpretación para el negocio:

«Ana debería recibir un email con "El otoño del patriarca" como primera recomendación, ya que: • Es de Gabriel García Márquez (autor que ya valoró positivamente)

• Tiene excelente valoración promedio (4.50/5)

• Es popular entre otros usuarios (2 préstamos)

• Score total: 82/100 puntos»

1.9.2 CASO 2: Análisis de morosidad y recuperación de deuda

Contexto:

El departamento de administración necesita un informe urgente de usuarios morosos para: • Enviar recordatorios automáticos

• Bloquear préstamos a usuarios con deuda > 20€

• Identificar patrones de morosidad (¿hay libros o autores asociados a más multas?)

Objetivo:

Informe ejecutivo de morosidad con acciones recomendadas.

Solución SQL:

```
-- Informe completo de morosidad
SELECT u.id_usuario,
        u.nombre_usuario,
        u.email,
        COUNT(DISTINCT m.id_multa) AS multas_pendientes,
        SUM(m.importe) AS deuda_total,
        MIN(p.fecha_prestamo) AS multa_mas_antigua,
        DATEDIFF(CURDATE(), MIN(p.fecha_prestamo)) AS dias_desde_primera_multa, --
        Clasificación de riesgo
        CASE
            WHEN SUM(m.importe) >= 30 THEN 'CRÍTICO '
            WHEN SUM(m.importe) >= 15 THEN 'ALTO '
            WHEN SUM(m.importe) >= 5 THEN 'MEDIO '
            ELSE 'BAJO '
        END AS nivel_riesgo,
        -- Acción recomendada
        CASE
            WHEN SUM(m.importe) >= 20 THEN 'BLOQUEAR CUENTA'
```

```
            WHEN SUM(m.importe) >= 10 THEN 'ENVIAR EMAIL URGENTE'
            ELSE 'ENVIAR RECORDATORIO'
        END AS accion_recomendada,
        -- Detalle de libros con multa
        GROUP_CONCAT(
            DISTINCT CONCAT(l.titulo, ' (', m.importe, '€)')
            ORDER BY m.importe DESC
            SEPARATOR ' | '
```

(continúe en la próxima página)

(proviene de la página anterior)

```
        ) AS detalle_libros_multados
FROM usuarios u
    INNER JOIN prestamos p ON u.id_usuario = p.id_usuario
    INNER JOIN multas m ON p.id_prestamo = m.id_prestamo
    INNER JOIN libros l ON p.id_libro = l.id_libro
WHERE m.pagada = FALSE
GROUP BY u.id_usuario, u.nombre_usuario, u.email
ORDER BY deuda_total DESC, dias_desde_primera_multa DESC;
```

Resultado esperado:

nombre_usuario	email	deuda_total	multas_pendientes	accion_recomendada	dias_desde_primera_multa
Carlos López	carlos.lopez@email.com	25.00	3	CRÍTICO 2/7	BLOQUEAR CUENTA	45
Pedro Sánchez	pedro.sanchez@email.com	12.50	2	ALTO 2/7	ENVIAR EMAIL URGENTE	20

Query adicional: Libros con más multas generadas (problema de gestión)

```
SELECT l.titulo,
        a.nombre_autor,
        COUNT(DISTINCT m.id_multa) AS multas_generadas,
        SUM(m.importe) AS importe_total_multas,
        COUNT(DISTINCT p.id_prestamo) AS total_prestamos,
        ROUND(COUNT(DISTINCT m.id_multa) / COUNT(DISTINCT p.id_prestamo) * 100, 2) AS porcentaje_multas
FROM libros l
    INNER JOIN autores a ON l.id_autor = a.id_autor
    INNER JOIN prestamos p ON l.id_libro = p.id_libro
    LEFT JOIN multas m ON p.id_prestamo = m.id_prestamo
GROUP BY l.id_libro, l.titulo, a.nombre_autor
HAVING multas_generadas > 0
ORDER BY porcentaje_multas DESC, multas_generadas DESC
LIMIT 5;
```

Interpretación para el negocio:

«Acción inmediata: Carlos López debe ser bloqueado (25€ de deuda, 45 días de antigüedad).
Problema detectado: Si un libro específico genera muchas multas, puede ser porque:

- Es muy popular pero tenemos pocas copias (aumentar inventario)
- Es un libro largo que requiere más tiempo (extender período de préstamo)»

1.9.3 CASO 3: Dashboard ejecutivo mensual

Contexto:

La directora de la biblioteca necesita un informe mensual automatizado que se envía por email el día 1 de cada mes con KPIs clave.

Objetivo:

Dashboard completo con métricas comparativas (mes actual vs mes anterior).

fi Solución SQL:

```
-- Definir variables de fechas
SET @mes_actual_inicio = DATE_FORMAT(CURDATE(), '%Y-%m-01');
SET @mes_actual_fin = LAST_DAY(@mes_actual_inicio);
SET @mes_anterior_inicio = DATE_SUB(@mes_actual_inicio, INTERVAL 1 MONTH); SET
@mes_anterior_fin = LAST_DAY(@mes_anterior_inicio);

-- KPI 1: Préstamos
SELECT 'PRÉSTAMOS' AS metrica,
        COUNT(CASE WHEN fecha_prestamo BETWEEN @mes_actual_inicio AND @mes_actual_fin␣
 ↪THEN 1 END) AS mes_actual,
        COUNT(CASE WHEN fecha_prestamo BETWEEN @mes_anterior_inicio AND @mes_anterior_ ↪fin
 THEN 1 END) AS mes_anterior,
        ROUND(
                (COUNT(CASE WHEN fecha_prestamo BETWEEN @mes_actual_inicio AND @mes_actual_ ↪fin
 THEN 1 END) -
                 COUNT(CASE WHEN fecha_prestamo BETWEEN @mes_anterior_inicio AND @mes_
 ↪anterior_fin THEN 1 END)) /
                NULLIF(COUNT(CASE WHEN fecha_prestamo BETWEEN @mes_anterior_inicio AND␣
 ↪@mes_anterior_fin THEN 1 END), 0) * 100
        , 2) AS variacion_porcentual
FROM prestamos

UNION ALL

-- KPI 2: Usuarios activos (que hicieron al menos 1 préstamo)
SELECT 'USUARIOS ACTIVOS' AS metrica,
        COUNT(DISTINCT CASE WHEN fecha_prestamo BETWEEN @mes_actual_inicio AND @mes_
 ↪actual_fin THEN id_usuario END) AS mes_actual,
        COUNT(DISTINCT CASE WHEN fecha_prestamo BETWEEN @mes_anterior_inicio AND @mes_
 ↪anterior_fin THEN id_usuario END) AS mes_anterior,
        ROUND(
                (COUNT(DISTINCT CASE WHEN fecha_prestamo BETWEEN @mes_actual_inicio AND␣
 ↪@mes_actual_fin THEN id_usuario END) -
                 COUNT(DISTINCT CASE WHEN fecha_prestamo BETWEEN @mes_anterior_inicio AND␣
 ↪@mes_anterior_fin THEN id_usuario END)) /
```

```
                NULLIF(COUNT(DISTINCT CASE WHEN fecha_prestamo BETWEEN @mes_anterior_
 inicio AND @mes_anterior_fin THEN id_usuario END), 0) * 100
            , 2) AS variacion_porcentual
FROM prestamos

UNION ALL
```

(continúe en la próxima página)

(proviene de la página anterior)

```
-- KPI 3: Valoraciones recibidas
SELECT 'VALORACIONES' AS metrica,
            COUNT(CASE WHEN fecha_valoracion BETWEEN @mes_actual_inicio AND @mes_actual_ fin
 THEN 1 END) AS mes_actual,
            COUNT(CASE WHEN fecha_valoracion BETWEEN @mes_anterior_inicio AND @mes_
 anterior_fin THEN 1 END) AS mes_anterior,
            ROUND(
                (COUNT(CASE WHEN fecha_valoracion BETWEEN @mes_actual_inicio AND @mes_
 actual_fin THEN 1 END) -
                 COUNT(CASE WHEN fecha_valoracion BETWEEN @mes_anterior_inicio AND @mes_
 anterior_fin THEN 1 END)) /
                NULLIF(COUNT(CASE WHEN fecha_valoracion BETWEEN @mes_anterior_inicio AND
 @mes_anterior_fin THEN 1 END), 0) * 100
            , 2) AS variacion_porcentual
FROM valoraciones

UNION ALL

-- KPI 4: Multas generadas (malo, queremos que baje)
SELECT 'MULTAS GENERADAS' AS metrica,
            COUNT(CASE WHEN p.fecha_prestamo BETWEEN @mes_actual_inicio AND @mes_actual_ fin
 THEN 1 END) AS mes_actual,
            COUNT(CASE WHEN p.fecha_prestamo BETWEEN @mes_anterior_inicio AND @mes_
 anterior_fin THEN 1 END) AS mes_anterior,
            ROUND(
                (COUNT(CASE WHEN p.fecha_prestamo BETWEEN @mes_actual_inicio AND @mes_
 actual_fin THEN 1 END) -
                 COUNT(CASE WHEN p.fecha_prestamo BETWEEN @mes_anterior_inicio AND @mes_
 anterior_fin THEN 1 END)) /
                NULLIF(COUNT(CASE WHEN p.fecha_prestamo BETWEEN @mes_anterior_inicio AND
 @mes_anterior_fin THEN 1 END), 0) * 100
            , 2) AS variacion_porcentual
FROM multas m
        INNER JOIN prestamos p ON m.id_prestamo = p.id_prestamo;
```

Query complementaria: Top 5 del mes

```
-- Top 5 libros más prestados del mes
SELECT 'TOP_LIBRO_MES' AS tipo,
            l.titulo AS item,
            a.nombre_autor AS detalle,
            COUNT(*) AS cantidad
FROM prestamos p
        INNER JOIN libros l ON p.id_libro = l.id_libro
        INNER JOIN autores a ON l.id_autor = a.id_autor
WHERE p.fecha_prestamo BETWEEN @mes_actual_inicio AND @mes_actual_fin GROUP
BY l.id_libro, l.titulo, a.nombre_autor
ORDER BY cantidad DESC
LIMIT 5

UNION ALL
```

```
-- Top 5 usuarios más activos del mes
SELECT 'TOP_USUARIO_MES' AS tipo,
        u.nombre_usuario AS item,
        u.email AS detalle,
        COUNT(*) AS cantidad
```

(continúe en la próxima página)

(proviene de la página anterior)

```
FROM prestamos p
    INNER JOIN usuarios u ON p.id_usuario = u.id_usuario
WHERE p.fecha_prestamo BETWEEN @mes_actual_inicio AND @mes_actual_fin GROUP
BY u.id_usuario, u.nombre_usuario, u.email
ORDER BY cantidad DESC
LIMIT 5;
```

Resultado esperado (formato email):

```
INFORME EJECUTIVO - NOVIEMBRE 2024
──────────────────────────────────────────────────────────────────────
INDICADORES CLAVE (vs Octubre 2024)
──────────────────────────────────────────────────────────────────────

• PRÉSTAMOS: 45 (+12.5% ↑)
• USUARIOS ACTIVOS: 18 (+5.9% ↑)
• VALORACIONES: 12 (+20.0% ↑)
• MULTAS GENERADAS: 3 (-25.0% ↓)

──────────────────────────────────────────────────────────────────────
TOP 5 LIBROS DEL MES
──────────────────────────────────────────────────────────────────────

1. Cien años de soledad (G. García Márquez) - 8 préstamos
2. 1984 (George Orwell) - 6 préstamos
3. El principito (A. Saint-Exupéry) - 5 préstamos
...
```

1.9.4 CASO 4: Detección de fraude y anomalías

Contexto:

El sistema ha detectado algunas inconsistencias:

- Libros prestados a varios usuarios simultáneamente (posible error de gestión)
- Usuarios con demasiados préstamos en poco tiempo (posible uso compartido de cuenta) • Valoraciones sospechosas (misma puntuación a muchos libros en el mismo día)

Objetivo:

Identificar anomalías para revisión manual.

Solución SQL:

```
-- ANOMALÍA 1: Libro prestado simultáneamente a múltiples usuarios SELECT
'PRÉSTAMO_SIMULTÁNEO' AS tipo_anomalia,
        l.titulo,
        GROUP_CONCAT(DISTINCT u.nombre_usuario ORDER BY u.nombre_usuario SEPARATOR ', ') AS
  usuarios_afectados,
        MIN(p.fecha_prestamo) AS fecha_inicio_conflicto,
        COUNT(DISTINCT p.id_usuario) AS usuarios_simultaneos,
        'REVISAR INVENTARIO' AS accion_sugerida
FROM libros l
    INNER JOIN prestamos p ON l.id_libro = p.id_libro
    INNER JOIN usuarios u ON p.id_usuario = u.id_usuario
WHERE p.devuelto = FALSE
GROUP BY l.id_libro, l.titulo
HAVING COUNT(DISTINCT p.id_usuario) > 1

UNION ALL

-- ANOMALÍA 2: Usuario con más de 5 préstamos en un día
SELECT 'PRÉSTAMOS_MASIVOS' AS tipo_anomalia,
        u.nombre_usuario AS titulo,
        DATE_FORMAT(p.fecha_prestamo, '%Y-%m-%d') AS usuarios_afectados,
        p.fecha_prestamo AS fecha_inicio_conflicto,
        COUNT(*) AS usuarios_simultaneos,
        'VERIFICAR CUENTA COMPARTIDA' AS accion_sugerida
FROM usuarios u
    INNER JOIN prestamos p ON u.id_usuario = p.id_usuario
GROUP BY u.id_usuario, u.nombre_usuario, DATE(p.fecha_prestamo), p.fecha_prestamo HAVING
COUNT(*) >= 5

UNION ALL

-- ANOMALÍA 3: Valoraciones sospechosas (≥5 en mismo día, todas misma puntuación) SELECT
'VALORACIONES_SOSPECHOSAS' AS tipo_anomalia,
        u.nombre_usuario AS titulo,
        DATE_FORMAT(v.fecha_valoracion, '%Y-%m-%d') AS usuarios_afectados,
        v.fecha_valoracion AS fecha_inicio_conflicto,
        COUNT(*) AS usuarios_simultaneos,
        CONCAT('VERIFICAR BOT - Todas puntuación ', v.puntuacion) AS accion_sugerida FROM usuarios u
    INNER JOIN valoraciones v ON u.id_usuario = v.id_usuario
GROUP BY u.id_usuario, u.nombre_usuario, DATE(v.fecha_valoracion), v.puntuacion, v.fecha_valoracion
HAVING COUNT(*) >= 5 AND COUNT(DISTINCT v.puntuacion) = 1;
```

1.9.5 CASO 5: Optimización de compras (qué libros adquirir)

Contexto:

La biblioteca tiene presupuesto para comprar 10 nuevos libros. Necesitan decidir qué comprar basándose en: • Demanda actual (libros con lista de espera)

• Tendencias (autores/géneros más solicitados)

• ROI esperado (libros que generarán más préstamos y valoraciones)

Base de datos a Datos: Especial Join

Objetivo:

Lista priorizada de compras recomendadas.

Solución SQL:

```
-- Análisis de demanda por autor
WITH demanda_autores AS (
    SELECT a.id_autor,
           a.nombre_autor,
           COUNT(DISTINCT p.id_prestamo) AS prestamos_totales,
           COUNT(DISTINCT l.id_libro) AS libros_en_catalogo,
           ROUND(COUNT(DISTINCT p.id_prestamo) / COUNT(DISTINCT l.id_libro), 2) AS␣
↪prestamos_por_libro,
           ROUND(AVG(v.puntuacion), 2) AS satisfaccion_promedio
    FROM autores a
        INNER JOIN libros l ON a.id_autor = l.id_autor
        LEFT JOIN prestamos p ON l.id_libro = p.id_libro
        LEFT JOIN valoraciones v ON l.id_libro = v.id_libro
    GROUP BY a.id_autor, a.nombre_autor
),

-- Libros actualmente más demandados (prestados recientemente)
libros_demanda_alta AS (
    SELECT l.id_libro,
           l.titulo,
               COUNT(DISTINCT p.id_prestamo) AS prestamos_ultimos_3_meses
    FROM libros l
        INNER JOIN prestamos p ON l.id_libro = p.id_libro
    WHERE p.fecha_prestamo >= DATE_SUB(CURDATE(), INTERVAL 3 MONTH)
    GROUP BY l.id_libro, l.titulo
    HAVING prestamos_ultimos_3_meses >= 3
)

-- Recomendación de compra
SELECT da.nombre_autor,
       da.libros_en_catalogo AS libros_actuales,
       da.prestamos_totales,
       da.prestamos_por_libro AS demanda_por_libro,
       da.satisfaccion_promedio,
       CASE
           WHEN da.prestamos_por_libro >= 5 AND da.satisfaccion_promedio >= 4.0 THEN
           'PRIORIDAD ALTA '
           WHEN da.prestamos_por_libro >= 3 AND da.satisfaccion_promedio >= 3.5 THEN
           'PRIORIDAD MEDIA '
           ELSE 'PRIORIDAD BAJA '
       END AS prioridad_compra,
       -- Score de decisión (0-100)
       ROUND(
           (da.prestamos_por_libro * 8) + -- Hasta 40 puntos por demanda (da.satisfaccion_promedio *
           12) + -- Hasta 60 puntos por␣
↪satisfacción
```

```
            CASE WHEN EXISTS ( -- Bonus si tiene libros en␣ ↪demanda alta
                SELECT 1 FROM libros_demanda_alta lda
                INNER JOIN libros l ON lda.id_libro = l.id_libro
```

(continúe en la próxima página)

(proviene de la página anterior)

```
                WHERE l.id_autor = da.id_autor
            ) THEN 10 ELSE 0 END
        , 0) AS score_compra,
        CONCAT(
            'Comprar ',
            GREATEST(1, FLOOR(da.prestamos_por_libro / da.libros_en_catalogo)), ' libro(s)
            adicional(es)'
        ) AS recomendacion
FROM demanda_autores da
WHERE da.prestamos_totales > 0 -- Solo autores con demanda comprobada ORDER BY
score_compra DESC, da.satisfaccion_promedio DESC
LIMIT 10;
```

Resultado esperado:

```
                    deman          sco
                    satisfac       recomen
nombre_autor li     priori
                    bros_actuales  cion_promedio  re_compra
                    da_por_libro   dad_compra     dacion
                                    3 6.33 4.50 PRIORIDAD ALTA 2/7 115 Comprar 2 libro(s)
Gabriel García Márquez
George Orwell 2 5.50 4.25 PRIORIDAD ALTA 2/7    95 Comprar 2 libro(s)
```

1.10 SECCIÓN 12: SCRIPT MySQL COMPLETO Y VERIFICADO

Esta sección incluye:

- ✅ Script de creación de base de datos
- ✅ Carga completa de datos de ejemplo
- ✅ Todos los 47 ejemplos del manual
- ✅ Tests automáticos de verificación
- ✅ Checklist final para el profesor

1.10.1 12.1. Script de creación de base de datos

```
-- ==========================================
-- SCRIPT DE CREACIÓN: BIBLIOTECA DIGITAL
-- Módulo: 0484 Bases de Datos
-- RA3: Consultas con JOINs
-- Profesor: David Valbuena Segura
-- ==========================================

-- Eliminar BD si existe (para poder re-ejecutar)
DROP DATABASE IF EXISTS biblioteca_digital;

-- Crear base de datos
CREATE DATABASE biblioteca_digital
```

(continúe en la próxima página)

(proviene de la página anterior)

```
CHARACTER SET utf8mb4
COLLATE utf8mb4_spanish_ci;

USE biblioteca_digital;

-- ==========================================
-- TABLA: AUTORES
-- ==========================================
CREATE TABLE autores (
    id_autor INT AUTO_INCREMENT PRIMARY KEY,
    nombre_autor VARCHAR(100) NOT NULL,
    nacionalidad VARCHAR(50),
    INDEX idx_nombre_autor (nombre_autor),
    INDEX idx_nacionalidad (nacionalidad)
) ENGINE=InnoDB;

-- ==========================================
-- TABLA: EDITORIALES
-- ==========================================
CREATE TABLE editoriales (
    id_editorial INT AUTO_INCREMENT PRIMARY KEY,
    nombre_editorial VARCHAR(100) NOT NULL,
    pais VARCHAR(50),
    INDEX idx_nombre_editorial (nombre_editorial)
) ENGINE=InnoDB;

-- ==========================================
-- TABLA: LIBROS
-- ==========================================
CREATE TABLE libros (
    id_libro INT AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(200) NOT NULL,
    id_autor INT NOT NULL,
    id_editorial INT NOT NULL,
    anio_publicacion YEAR,
    isbn VARCHAR(20),
    FOREIGN KEY (id_autor) REFERENCES autores(id_autor),
    FOREIGN KEY (id_editorial) REFERENCES editoriales(id_editorial),
    INDEX idx_titulo (titulo),
    INDEX idx_autor (id_autor),
    INDEX idx_editorial (id_editorial),
    INDEX idx_anio (anio_publicacion)
) ENGINE=InnoDB;

-- ==========================================
-- TABLA: USUARIOS
-- ==========================================
CREATE TABLE usuarios (
    id_usuario INT AUTO_INCREMENT PRIMARY KEY,
    nombre_usuario VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    fecha_registro DATE NOT NULL,
    INDEX idx_nombre_usuario (nombre_usuario),
    INDEX idx_email (email),
    INDEX idx_fecha_registro (fecha_registro)
) ENGINE=InnoDB;
```

(continúe en la próxima página)

(proviene de la página anterior)

```
-- ==========================================
-- TABLA: PRESTAMOS
-- ==========================================
CREATE TABLE prestamos (
    id_prestamo INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario INT NOT NULL,
    id_libro INT NOT NULL,
    fecha_prestamo DATE NOT NULL,
    fecha_devolucion DATE,
    devuelto BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario),
    FOREIGN KEY (id_libro) REFERENCES libros(id_libro),
    INDEX idx_usuario (id_usuario),
    INDEX idx_libro (id_libro),
    INDEX idx_fecha_prestamo (fecha_prestamo),
    INDEX idx_devuelto (devuelto),
    INDEX idx_usuario_devuelto (id_usuario, devuelto)
) ENGINE=InnoDB;

-- ==========================================
-- TABLA: VALORACIONES
-- ==========================================
CREATE TABLE valoraciones (
    id_valoracion INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario INT NOT NULL,
    id_libro INT NOT NULL,
    puntuacion INT NOT NULL CHECK (puntuacion BETWEEN 1 AND 5),
    comentario TEXT,
    fecha_valoracion DATE NOT NULL,
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario),
    FOREIGN KEY (id_libro) REFERENCES libros(id_libro),
    INDEX idx_usuario (id_usuario),
    INDEX idx_libro (id_libro),
    INDEX idx_puntuacion (puntuacion),
    INDEX idx_fecha_valoracion (fecha_valoracion),
    UNIQUE KEY unique_usuario_libro (id_usuario, id_libro)
) ENGINE=InnoDB;

-- ==========================================
-- TABLA: MULTAS
-- ==========================================
CREATE TABLE multas (
    id_multa INT AUTO_INCREMENT PRIMARY KEY,
    id_prestamo INT NOT NULL,
    importe DECIMAL(10,2) NOT NULL,
    pagada BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (id_prestamo) REFERENCES prestamos(id_prestamo),
    INDEX idx_prestamo (id_prestamo),
    INDEX idx_pagada (pagada)
) ENGINE=InnoDB;
```

1.10.2 12.2. Script de carga de datos

```
-- ============================================
-- CARGA DE DATOS: BIBLIOTECA DIGITAL
-- ============================================

-- ============================================
-- INSERTAR AUTORES
-- ============================================
INSERT INTO autores (nombre_autor, nacionalidad) VALUES
 ('Gabriel García Márquez', 'Colombiano'),
 ('George Orwell', 'Británico'),
 ('Miguel de Cervantes', 'Español'),
 ('Antoine de Saint-Exupéry', 'Francés'),
 ('Fiódor Dostoyevski', 'Ruso'),
 ('Franz Kafka', 'Checo'),
 ('Julio Cortázar', 'Argentino'),
 ('Isabel Allende', 'Chilena'),
 ('Mario Vargas Llosa', 'Peruano');

-- ============================================
-- INSERTAR EDITORIALES
-- ============================================
INSERT INTO editoriales (nombre_editorial, pais) VALUES
 ('Editorial Sudamericana', 'Argentina'),
 ('Gallimard', 'Francia'),
 ('Secker & Warburg', 'Reino Unido'),
 ('Francisco de Robles', 'España'),
 ('Editorial Planeta', 'España');

-- ============================================
-- INSERTAR LIBROS
-- ============================================
INSERT INTO libros (titulo, id_autor, id_editorial, anio_publicacion, isbn) VALUES ('Cien años de soledad', 1,
 1, 1967, '978-0307474728'),
 ('El amor en los tiempos del cólera', 1, 1, 1985, '978-0307389732'), ('El otoño del patriarca',
 1, 1, 1975, '978-8497592444'),
 ('1984', 2, 3, 1949, '978-0451524935'),
 ('Rebelión en la granja', 2, 3, 1945, '978-0451526342'),
 ('Don Quijote de la Mancha', 3, 4, 1605, '978-8424936471'),
 ('El principito', 4, 2, 1943, '978-0156012195'),
 ('Crimen y castigo', 5, 5, 1866, '978-0486415871'),
 ('La metamorfosis', 6, 5, 1915, '978-0486290300'),
 ('Rayuela', 7, 1, 1963, '978-8437604572');

-- ============================================
-- INSERTAR USUARIOS
-- ============================================
INSERT INTO usuarios (nombre_usuario, email, fecha_registro) VALUES ('Ana García',
 'ana.garcia@email.com', '2024-01-15'),
 ('Carlos López', 'carlos.lopez@email.com', '2024-02-20'),
 ('María Fernández', 'maria.fernandez@email.com', '2024-03-10'),
 ('Pedro Sánchez', 'pedro.sanchez@email.com', '2024-04-05'),
```

```
('David Ruiz', 'david.ruiz@email.com', '2024-05-12'),
('Elena Moreno', 'elena.moreno@email.com', '2024-06-18'),
('Luis Martínez', 'luis.martinez@email.com', '2024-07-22');
```

(continúe en la próxima página)

(proviene de la página anterior)

```
-- ==========================================
-- INSERTAR PRÉSTAMOS
-- ==========================================
INSERT INTO prestamos (id_usuario, id_libro, fecha_prestamo, fecha_devolucion, devuelto) VALUES
-- Ana García (id=1) - Usuario activo
(1, 1, '2024-10-01', '2024-10-15', TRUE),
(1, 4, '2024-10-20', NULL, FALSE),
(1, 7, '2024-11-01', '2024-11-10', TRUE),
(1, 2, '2024-11-15', NULL, FALSE),

-- Carlos López (id=2) - Usuario con multas
(2, 2, '2024-09-15', '2024-10-20', TRUE), -- Entrega tardía
(2, 5, '2024-10-05', NULL, FALSE), -- Aún sin devolver (retrasado) (2, 8, '2024-11-01', NULL,
FALSE),

-- María Fernández (id=3)
(3, 3, '2024-10-10', '2024-10-20', TRUE),
(3, 6, '2024-10-25', '2024-11-05', TRUE),
(3, 9, '2024-11-10', NULL, FALSE),

-- Pedro Sánchez (id=4) - Usuario con multas
(4, 1, '2024-09-20', '2024-10-25', TRUE), -- Entrega tardía
(4, 10, '2024-11-05', NULL, FALSE),

-- David Ruiz (id=5)
(5, 4, '2024-10-12', '2024-10-22', TRUE),
(5, 7, '2024-11-08', NULL, FALSE),

-- Elena Moreno (id=6)
(6, 5, '2024-10-18', '2024-10-28', TRUE),
(6, 2, '2024-11-12', NULL, FALSE),

-- Luis Martínez (id=7) - Sin préstamos recientes activos
(7, 8, '2024-09-05', '2024-09-15', TRUE);

-- ==========================================
-- INSERTAR VALORACIONES
-- ==========================================
INSERT INTO valoraciones (id_usuario, id_libro, puntuacion, comentario, fecha_valoracion) VALUES
-- Ana García
(1, 1, 5, 'Obra maestra de García Márquez', '2024-10-16'),
(1, 7, 5, 'Un clásico imprescindible', '2024-11-11'),

-- Carlos López
(2, 2, 4, 'Muy buena historia de amor', '2024-10-21'),
(2, 5, 3, 'Interesante pero esperaba más', '2024-11-15'),

-- María Fernández
(3, 3, 5, 'García Márquez en su mejor momento', '2024-10-21'),
(3, 6, 4, 'Una joya de la literatura española', '2024-11-06'),

-- Pedro Sánchez
(4, 1, 4, 'Muy buena novela', '2024-10-26'),
```

```
-- David Ruiz
```

(continúe en la próxima página)

(proviene de la página anterior)

```
(5, 4, 5, 'Distopía brillante y vigente', '2024-10-23'),
(5, 7, 4, 'Emotivo y profundo', '2024-11-20'),

-- Elena Moreno
(6, 5, 2, 'No me convenció', '2024-10-29'),

-- Luis Martínez
(7, 8, 5, 'Dostoyevski es un genio', '2024-09-16');

-- ============================================
-- INSERTAR MULTAS
-- ============================================
INSERT INTO multas (id_prestamo, importe, pagada) VALUES
-- Carlos López - 2 multas pendientes
(5, 12.50, FALSE), -- Préstamo id=5 (libro 2) - Retraso de 5 días (6, 8.00, FALSE), --
Préstamo id=6 (libro 5) - Aún sin devolver

-- Pedro Sánchez - 1 multa pendiente
(11, 10.00, FALSE), -- Préstamo id=11 (libro 1) - Retraso de 10 días

-- Multas ya pagadas
(5, 5.00, TRUE); -- Multa anterior de Carlos pagada
```

1.10.3 12.3. Tests de verificación automática

```
-- ============================================
-- TESTS DE VERIFICACIÓN
-- Estos tests deben ejecutarse después de
-- crear la BD y cargar los datos
-- ============================================

SELECT '============================================' AS '';
SELECT 'INICIANDO TESTS DE VERIFICACIÓN' AS '';
SELECT '============================================' AS '';
SELECT '' AS '';

-- TEST 1: Verificar que todas las tablas existen
SELECT '✓ TEST 1: Verificación de tablas' AS resultado;
SELECT COUNT(*) AS tablas_creadas,
        CASE
            WHEN COUNT(*) = 7 THEN ' OK - 7 tablas creadas'
            ELSE ' ERROR - Faltan tablas'
        END AS estado
FROM information_schema.tables
WHERE table_schema = 'biblioteca_digital';
SELECT '' AS '';

-- TEST 2: Verificar datos cargados
SELECT '✓ TEST 2: Verificación de datos' AS resultado;
SELECT
    (SELECT COUNT(*) FROM autores) AS autores,
    (SELECT COUNT(*) FROM editoriales) AS editoriales,
    (SELECT COUNT(*) FROM libros) AS libros,
```

(continúe en la próxima página)

(proviene de la página anterior)

```
        (SELECT COUNT(*) FROM usuarios) AS usuarios,
        (SELECT COUNT(*) FROM prestamos) AS prestamos,
        (SELECT COUNT(*) FROM valoraciones) AS valoraciones,
        (SELECT COUNT(*) FROM multas) AS multas,
       CASE
            WHEN (SELECT COUNT(*) FROM autores) = 9
              AND (SELECT COUNT(*) FROM libros) = 10
              AND (SELECT COUNT(*) FROM usuarios) = 7
            THEN ' OK - Datos cargados correctamente'
            ELSE ' ERROR - Faltan datos'
       END AS estado;
SELECT '' AS '';

-- TEST 3: Verificar integridad referencial
SELECT '✓ TEST 3: Integridad referencial' AS resultado;
SELECT
       COUNT(*) AS libros_sin_autor_valido
FROM libros l
LEFT JOIN autores a ON l.id_autor = a.id_autor
WHERE a.id_autor IS NULL;

SELECT CASE
       WHEN (SELECT COUNT(*) FROM libros l LEFT JOIN autores a ON l.id_autor = a.id_autor WHERE
  a.id_autor IS NULL) = 0
       THEN ' OK - Todas las FK válidas'
       ELSE ' ERROR - Hay registros huérfanos'
END AS estado;
SELECT '' AS '';

-- TEST 4: Verificar ejemplos del manual
SELECT '✓ TEST 4: Ejemplos del manual' AS resultado;

-- Ejemplo 1: INNER JOIN básico (debe devolver 10 libros)
SET @resultado_ej1 = (
       SELECT COUNT(*)
       FROM libros l
       INNER JOIN autores a ON l.id_autor = a.id_autor
);

SELECT CASE
       WHEN @resultado_ej1 = 10
       THEN ' OK - Ejemplo 1 (INNER JOIN básico)'
       ELSE CONCAT(' ERROR - Ejemplo 1. Esperado: 10, Obtenido: ', @resultado_ej1) END AS estado;

-- Ejemplo 10: LEFT JOIN (debe incluir usuario sin préstamos)
SET @resultado_ej10 = (
       SELECT COUNT(DISTINCT u.id_usuario)
       FROM usuarios u
       LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario
);

SELECT CASE
       WHEN @resultado_ej10 = 7
       THEN ' OK - Ejemplo 10 (LEFT JOIN con NULL)'
       ELSE CONCAT(' ERROR - Ejemplo 10. Esperado: 7, Obtenido: ', @resultado_ej10) END AS estado;
```

(continúe en la próxima página)

(proviene de la página anterior)

```
-- Ejemplo 13: Usuarios sin préstamos (debe devolver al menos 1)
SET @resultado_ej13 = (
    SELECT COUNT(*)
    FROM usuarios u
    LEFT JOIN prestamos p ON u.id_usuario = p.id_usuario
    WHERE p.id_prestamo IS NULL
);

SELECT CASE
    WHEN @resultado_ej13 >= 0
    THEN CONCAT(' OK - Ejemplo 13 (LEFT JOIN + IS NULL): ', @resultado_ej13, '␣ ↪usuario(s) sin
 préstamos')
    ELSE ' ERROR - Ejemplo 13'
END AS estado;

-- Ejemplo 32: COUNT con agregaciones
SET @resultado_ej32 = (
    SELECT COUNT(DISTINCT u.id_usuario)
    FROM usuarios u
    INNER JOIN prestamos p ON u.id_usuario = p.id_usuario
);

SELECT CASE
    WHEN @resultado_ej32 > 0
    THEN CONCAT(' OK - Ejemplo 32 (Agregaciones): ', @resultado_ej32, ' usuarios con␣ ↪préstamos')
    ELSE ' ERROR - Ejemplo 32'
END AS estado;

SELECT '' AS '';

-- TEST 5: Verificar multas y préstamos
SELECT '✓ TEST 5: Verificación de multas' AS resultado;
SELECT
    (SELECT COUNT(*) FROM multas WHERE pagada = FALSE) AS multas_pendientes,
    (SELECT SUM(importe) FROM multas WHERE pagada = FALSE) AS deuda_total, CASE
        WHEN (SELECT COUNT(*) FROM multas WHERE pagada = FALSE) = 3
         AND (SELECT SUM(importe) FROM multas WHERE pagada = FALSE) = 30.50 THEN '
        OK - Multas correctas (3 pendientes, 30.50€ total)'
        ELSE ' ERROR - Multas incorrectas'
    END AS estado;
SELECT '' AS '';

-- TEST 6: Verificar valoraciones
SELECT '✓ TEST 6: Verificación de valoraciones' AS resultado;
SELECT
    COUNT(*) AS total_valoraciones,
    ROUND(AVG(puntuacion), 2) AS puntuacion_promedio,
    MIN(puntuacion) AS min_puntuacion,
    MAX(puntuacion) AS max_puntuacion,
    CASE
        WHEN COUNT(*) = 11
         AND MIN(puntuacion) >= 1
         AND MAX(puntuacion) <= 5
        THEN ' OK - Valoraciones válidas'
```

(continúe en la próxima página)

(proviene de la página anterior)

```
            ELSE ' ERROR - Valoraciones fuera de rango'
        END AS estado
FROM valoraciones;
SELECT '' AS '';

-- TEST 7: Verificar índices
SELECT '✓ TEST 7: Verificación de índices' AS resultado;
SELECT
        COUNT(DISTINCT index_name) AS indices_totales,
        CASE
            WHEN COUNT(DISTINCT index_name) >= 20
            THEN ' OK - Índices creados'
            ELSE ' AVISO - Pocos índices (rendimiento podría mejorar)'
        END AS estado
FROM information_schema.statistics
WHERE table_schema = 'biblioteca_digital'
    AND index_name != 'PRIMARY';
SELECT '' AS '';

SELECT '==========================================' AS '';
SELECT ' TESTS COMPLETADOS' AS '';
SELECT '==========================================' AS '';
```